Programación con TypeScript

CESUR

Tu Centro Oficial de FP

1. Introducción a TypeScript

TYPESCRIPT - es una super extensión de Javascript que le agrega funcionalidades adicionales, pero comparte la misma base y funcionamiento.

Cualquier código JavaScript válido también es válido en **TypeScript**.

Entre las funcionalidades que **TypeScript** añade a JavaScript, principalmente tenemos,

1. **Tipado estático** (de ahí su nombre)
2. **Mejoras en la programación orientada a objetos**

1. TIPADO ESTÁTICO

En JavaScript los tipos se determinan en tiempo de ejecución.

TypeScript nos permite definir los tipos de variables, parámetros y retornos de funciones en tiempo de desarrollo.

```
function suma(a: number, b: number): number {
    return a + b;
}
```

suma es una función que acepta dos parámetros de tipo **number** y retorna un **number.**

Si intentamos pasar un string como argumento, **TypeScript** lanzará un error en tiempo de compilación.

```
suma("1", "2");      No se puede asignar un argumento de tipo "string" al parámetro de tipo "number".
```

2. PROGRAMACIÓN ORIENTADA A OBJETOS

TypeScript introduce conceptos de la programación orientada a objetos (POO) como clases, interfaces y herencia, que no son nativos en JavaScript.

Estas características nos ayudan a organizar y estructurar del código, especialmente en proyectos grandes.

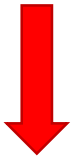
```
class Animal {
    constructor(public nombre: string) {}

    hacerSonido(): void {
        console.log(`${this.nombre} hace un sonido.`);
    }
}
class Perro extends Animal {
    ladrar(): void {
        console.log(`${this.nombre} ladra.`);
    }
}
const miPerro = new Perro('Firulais');
miPerro.hacerSonido(); // Firulais hace un sonido.
miPerro.ladrar(); // Firulais ladra.
```

COMPATIBILIDAD CON JAVASCRIPT

TypeScript es un superconjunto de JavaScript, es decir, es una capa puesta por encima de JavaScript.

Por tanto, todo código JavaScript válido es también código TypeScript válido. Esto nos permite migrar gradualmente un proyecto JavaScript existente a TypeScript (sin tener que reescribir todo el código desde cero).

```
// Código JavaScript válido
function saludar(nombre) {
    return `Hola, ${nombre}`;
}
```



```
// Lo podemos ir convirtiendo gradualmente a TypeScript
function saludar(nombre: string): string {
    return `Hola, ${nombre}`;
}
```

INSTALACIÓN DE TYPESCRIPT

Node.js es un entorno de ejecución de JavaScript que nos permite ejecutar JavaScript en el lado del servidor.

NPM (Node Package Manager) es el gestor de paquetes que se incluye con Node.js.

Necesitaremos ambos para instalar TypeScript y ejecutar nuestros programas.

Procedimiento:

1. **Instalar Node.js**
2. **Verificar la instalación de Node.js y npm**: Abre una terminal y ejecutar los siguientes comandos:

```
node -v npm -v
```

CÓMO INSTALAR TYPESCRIPT

Para instalar TypeScript, en la terminal de tu SO ejecuta el siguiente comando:

```
npm install -g typescript
```

NOTA: El modificador -g asegura que TypeScript se instale globalmente (lo que significa que estará disponible en cualquier proyecto de tu máquina).

CÓMO ACTUALIZAR TYPESCRIPT

Para actualizar TypeScript a la última versión disponible, abrimos el terminal y ejecutamos el siguiente comando:

```
npm update -g typescript
```

3. Primer proyecto

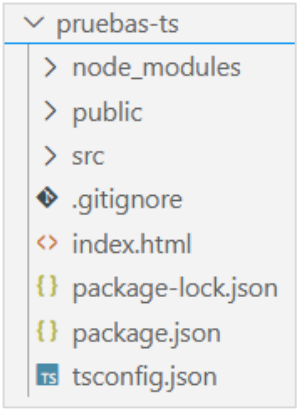
1) Abrimos Visual Studio Code: *Archivo – Abrir carpeta – pruebas-ts*

2) Abrir la terminal para crear nuestro proyecto de React con TypeScript

3) Teclear ***npm create vite@latest***

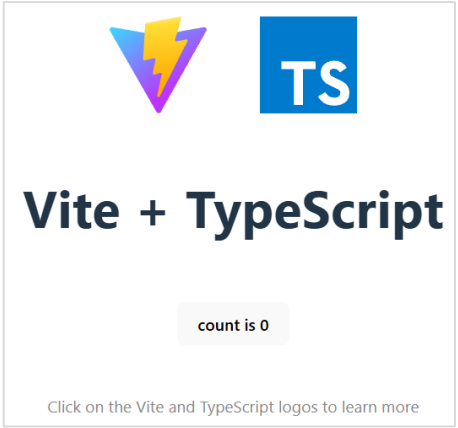
- Framework : ***Vanilla***
- Variante: ***TypeScript***

4) Cuando se crea el proyecto tiene esta estructura:



5) Haciendo clic en el servidor local nos muestra la página:

```
VITE v7.3.1  ready in 242 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



IMPORTANTE: IR A LA CARPETA DEL PROYECTO, para no tener que cambiar continuamente de directorio. (Archivo – Abrir Carpeta – REACT – pruebas-ts)

3. Primer proyecto

Para crear la primera página en la carpeta src, creamos un directorio llamado bases y dentro index.ts

Escribir el siguiente código

```
function saludar(nombre: string): string {
    return `Hola, ${nombre}!`;
}

const nombre = "Luis";
console.log(saludar(nombre));
```

Este código,

- Define una función saludar que toma un nombre como parámetro y devuelve una cadena de saludo
- Luego, define una constante nombre con valor “Luis”
- Llama a console.log para imprimir el saludo en la consola

Para poder ejecutar el proyecto:

1) En index.html se debe llamar a main (por defecto)

```
<script type="module" src="/src/main.ts"></script>
```

2) En main debemos enlazar a archivo index.ts

```
import './bases/index'
```

EL FICHERO TSCONFIG.JSON

El archivo **tsconfig.json** es el archivo de configuración principal de un proyecto TypeScript.

Es un fichero JSON que contiene opciones y parámetros que controlan el comportamiento de TypeScript en nuestro proyecto.

```
{ {2}
  compilerOptions : { {6}
      target : "es2016"
      module : "commonjs"
      esModuleInterop : true
      forceConsistentCasingInFileNames : true
      strict : true
      skipLibCheck : true
    }
  include : [ [1]
      0 : "src"
    ]
}
```

5. Módulos en TypeScript

MÓDULO - en TypeScript es un archivo que contiene código y puede exportar a otros ficheros partes de ese código para que sean utilizadas en otros archivos.

Esto nos permite dividir nuestro código en módulos más pequeños y mantener una estructura organizada y modular en nuestro proyecto.

Para crear un módulo en TypeScript, simplemente debemos utilizar la palabra clave **export** antes de la declaración de una clase, función, constante o variable que deseamos hacer accesible desde otros archivos.

```typescript
// archivo: utils.ts
export function sum(a: number, b: number): number {
  return a + b;
}

export const PI = 3.1416;

export default class Person {
  constructor(public name: string) {}
}
```

En este código:

- Creamos un módulo llamado utils.ts
- Este exporta una clase Person por defecto
- Exporta una función sum y una constante Pi con nombre.

5. Módulos en TypeScript

Podemos exportar una única entidad por defecto desde un módulo utilizando la palabra clave **export default**.

```
export default class MiClase {
  // Código de la clase
}
```

Para importar la entidad exportada por defecto, simplemente utilizamos el nombre que queremos asignarle en nuestra importación.

```
import MiClase from './ruta/al/modulo';
```

La importación de módulos nos permite utilizar el código exportado desde otro archivo en nuestro propio archivo.

Para importar un módulo en TypeScript, utilizamos la palabra clave **import** seguida del nombre del módulo que queremos importar.

```
import { nombreModulo } from './ruta/al/modulo';
```

Para utilizar los elementos exportados desde un módulo en otro archivo TypeScript, debemos importarlos. La sintaxis para importar un módulo es la siguiente:

```
import Person from './utils';
```

6. Namespaces en TypeScript

NAMESPACES (espacios de nombres) - son una forma de organizar y agrupar nuestro código en módulos lógicos para evitar colisiones de nombres entre diferentes partes de nuestro programa.

Un espacio de nombres en Typescript encapsula variables, funciones, clases y otros elementos en un contenedor. Esto nos permite agrupar y organizar nuestro código, evitando conflictos y mejorando la legibilidad y mantenibilidad del mismo.

Para definir un espacio de nombres en Typescript, utilizamos la palabra clave **namespace** seguida del nombre que deseamos asignarle al espacio.

```
namespace MiEspacio {
  export function saludar() {
    console.log("¡Hola desde MiEspacio!");
  }
}
```

En este ejemplo:

- Creamos un espacio de nombres llamado MiEspacio
- Este contiene una función saludar.
- La declaración export permite que la función sea accesible fuera del espacio de nombres.

6. Namespaces en TypeScript

Una vez que hemos definido un espacio de nombres, podemos utilizarlo en otras partes de nuestro programa. Para acceder a los elementos dentro de un espacio de nombres, utilizamos la sintaxis ***NombreEspacio.NombreElemento***.

```
MiEspacio.saludar();
```

En este caso, estamos llamando a la función saludar que se encuentra dentro del espacio de nombres MiEspacio.

En Typescript, también es posible **anidar espacios de nombres** dentro de otros espacios de nombres. Esto nos permite organizar aún más nuestro código y evitar colisiones de nombres. Por ejemplo:

```
namespace MiEspacio {
  export namespace SubEspacio {
    export function despedir() {
      console.log("¡Adiós desde SubEspacio!");
    }
  }
}
```

- Hemos anidado un espacio de nombres llamado SubEspacio dentro del espacio de nombres MiEspacio
- La función despedir se encuentra dentro del espacio de nombres SubEspacio
- Para acceder a esta función, utilizamos la sintaxis MiEspacio.SubEspacio.despedir()

7. Variables en TypeScript

En TypeScript, podemos declarar variables utilizando la palabra clave **let** o **const**, seguida del nombre de la variable (opcionalmente, su tipo).

let - declara variables que tienen un ámbito de bloque (block scope). Esto significa que la variable solo es accesible dentro del bloque donde las definimos.

```
let nombre: string = "Luis";
nombre = "Carlos"; // Correcto
console.log(nombre); // Carlos
```

const - se utiliza para declarar constantes (es decir, variables cuyo valor no cambiará una vez asignado). También tienen un ámbito de bloque.

```
const PI: number = 3.1416;
console.log(PI); // 3.1416

// PI = 3.14; // Error: No se puede asignar un nuevo valor a una constante
```

TIPOS DE VARIABLES

En TypeScript, podemos asignar diferentes tipos a nuestras variables. Por ejemplo:

```
let nombre: string = "Luis";
let edad: number = 25;
let esMayor: boolean = true;
let listaNumeros: number[] = [1, 2, 3, 4, 5];
let persona: { nombre: string, edad: number } = { nombre: "Luis", edad: 25 };
```

7. Variables en TypeScript

ÁMBITO DE UNA VARIABLE

Determina dónde se puede acceder a ella en el código.

TypeScript tiene tres tipos principales de ámbito:

1) **ÁMBITO GLOBAL** - Una variable declarada fuera de cualquier función o bloque tiene un ámbito global y es accesible desde cualquier parte del código.

```
let globalVar: string = "Soy global";

function mostrarGlobal() {
    console.log(globalVar); // Soy global
}
```

2) **ÁMBITO DE BLOQUE** - Las variables declaradas con **let** o **const** dentro de un bloque (por ejemplo, dentro de una estructura if, for, o {}) solo son accesibles dentro de ese bloque.

```
if (true) {
    let bloqueLet: string = "Dentro del bloque";
    console.log(bloqueLet); // Dentro del bloque
}
// console.log(bloqueLet); // Error: bloqueLet no está definido
```

3) **ÁMBITO DE FUNCIÓN** - Una variable declarada dentro de una función solo es accesible dentro de esa función. Las variables declaradas con **var** tienen ámbito de función. NO ES RECOMENDABLE SU USO

```
function ejemploFuncion() {
    var funcionVar: string = "Dentro de la función";
    console.log(funcionVar); // Dentro de la función
}
// console.log(funcionVar); // Error: funcionVar no está definido
```

7. Variables en TypeScript

INFERENCIA DE TIPOS

En TypeScript, el compilador puede inferir automáticamente el tipo de una variable según el valor que le asignemos.

Esto significa que no siempre es necesario especificar el tipo de una variable explícitamente, ya que TypeScript puede deducirlo por sí mismo.

```
let nombre = "Luis";
let edad = 25;
```

No hemos especificado el tipo de las variables nombre y edad

Sin embargo, TypeScript puede inferir que nombre es de tipo string y edad es de tipo number por los valores que les asignamos.

TypeScript también infiere en casos más complejos, como los tipos de retorno de las funciones basándose en el valor de retorno.

```
function sumar(a: number, b: number) {
    return a + b; // Tipo inferido como number
}

let resultado = sumar(1, 2); // Tipo inferido como number
```

8. Tipos de datos en TypeScript

Los tipos en TypeScript son la herramienta básica para definir las características y el comportamiento de las variables y objetos en el código.

Los tipos nos permiten especificar qué tipo de valores puede contener una variable, qué tipo de argumentos puede recibir una función, y qué tipo de valor devolverá una función (entre otros).

El sistema de tipos en TypeScript es un poco "especial" frente a otros lenguajes.

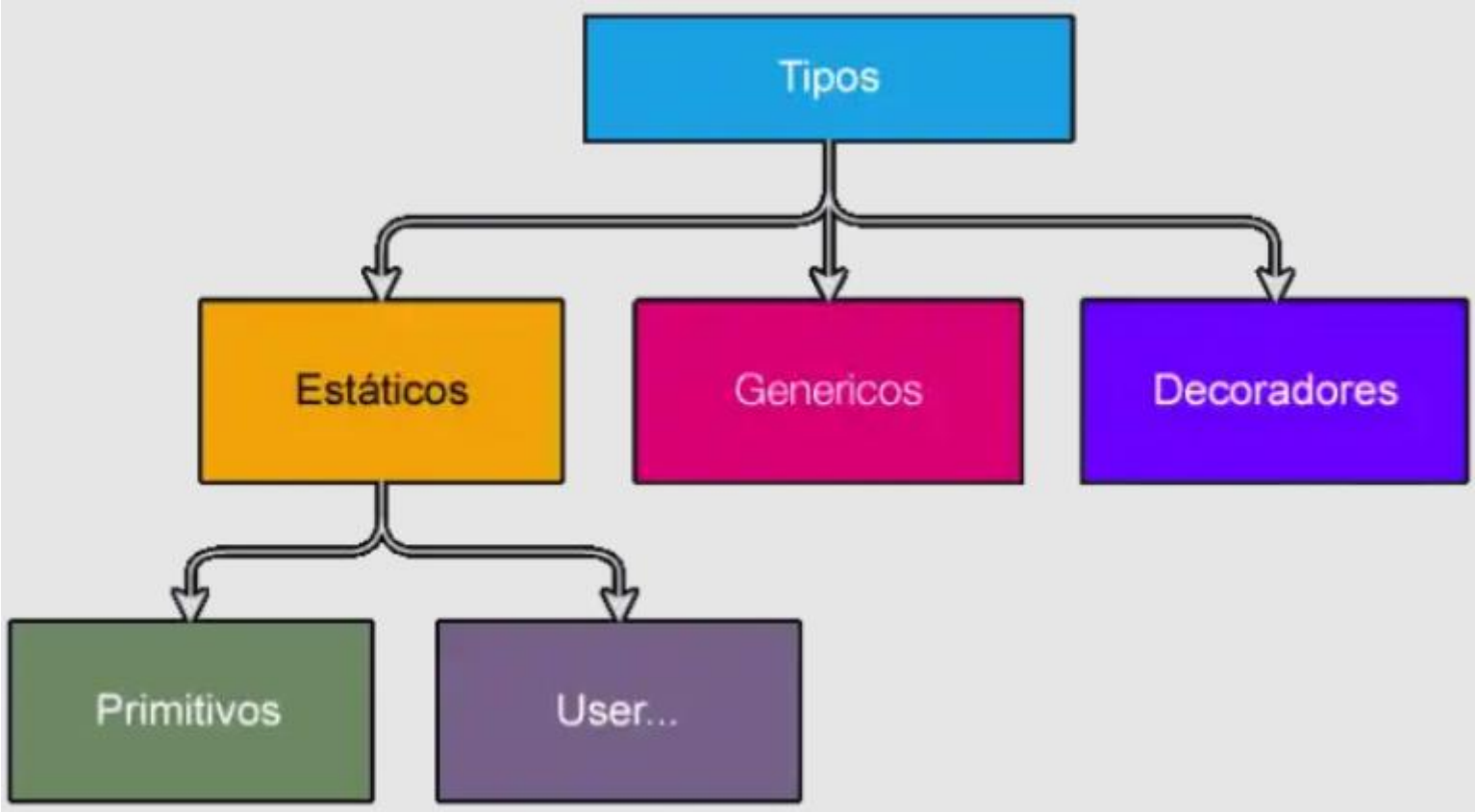
Algunas de las características que lo hacen especial son:

- **Estática:** La tipificación se verifica en tiempo de compilación, lo que permite detectar errores antes de ejecutar el código.
- **Opcional:** Aunque TypeScript es un lenguaje fuertemente tipado, la tipificación es opcional y puede ser progresivamente añadida a un proyecto.
- **Estructural:** TypeScript utiliza una tipificación estructural, donde dos tipos son compatibles si tienen la misma forma o estructura

8. Tipos de datos en TypeScript

CATEGORÍAS DE TIPOS EN TYPESCRIPT

Los tipos en TypeScript se puede dividir en varias categorías principales:



8. Tipos de datos en TypeScript

1) Tipos Estáticos - Son los tipos que se verifican en tiempo de compilación. Incluyen:

- **Tipos Primitivos** - Los tipos básicos que se encuentran en casi todos los lenguajes de programación.
- **Tipos Definidos por el Usuario** (User Defined Types) - Tipos que el usuario define para satisfacer necesidades específicas de la aplicación.

2) Tipos Genéricos - Permiten la definición de componentes que pueden trabajar con diferentes tipos sin especificar de antemano el tipo exacto. Los genéricos proporcionan una manera de crear funciones y clases reutilizables.

3) Decoradores - Son una forma especial de declaración que se puede aplicar a clases, métodos, propiedades y parámetros. Los decoradores permiten añadir metadatos y modificar el comportamiento de las clases y sus miembros.

8. Tipos de datos en TypeScript

TIPOS DE DATOS ESTÁTICOS I

Los tipos primitivos en TypeScript habituales:

TIPO	DESCRIPCIÓN
`number`	Representa tanto números enteros como de punto flotante
`bigint`	Representa números enteros grandes
`string`	Representa cadenas de texto
`boolean`	Representa valores lógicos: true o false
`symbol`	Representa un valor único y mutable

TIPOS DE DATOS ESTÁTICOS II

Otros tipos primitivos que tienen un uso especial

TIPO	DESCRIPCIÓN
`any`	Representa un tipo compatible con todos los demás
`void`	Representa la ausencia de un valor de retorno
`undefined`	Representa una variable que ha sido declarada pero no inicializada
`null`	Representa la ausencia de un valor
`never`	Representa el tipo de valor que nunca ocurre

8. Tipos de datos en TypeScript

EJEMPLOS DE TIPOS DE DATOS BÁSICOS

1) **number** - números enteros o decimales

```
let edad: number = 25;
```

2) **string** – texto

```
let nombre: string = "Luis";
```

3) **boolean** – valores verdadero o falso

```
let esMayorDeEdad: boolean = true;
```

4) **array** – múltiples valores del mismo tipo en una variable

```
let numeros: number[] = [1, 2, 3, 4, 5];
```

8. Tipos de datos en TypeScript

TIPOS DE DATOS ESPECIALES

1) **any** – permite almacenar cualquier tipo de valor (se usa cuando no conocemos el tipo de dato de una variable)

```
let variable: any = "Hola";
variable = 10;
```

2) **void** – representa la usencia de un tipo. Se utiliza en funciones que no retornan ningún valor

```
function saludar(): void {
  console.log("Hola");
}
```

3) **null y undefined** – representan la ausencia de un valor

```
let nulo: null = null;
let indefinido: undefined = undefined;
```

4) **never** – representa valores que nunca ocurren. Se utiliza en funciones que lanzan excepciones o entran en bucles infinitos

```
function lanzarError(): never {
  throw new Error("Ocurrió un error");
}
```

8. Tipos de datos en TypeScript

TIPOS DEFINIDOS POR EL USUARIO (PERSONALIZADOS)

TIPO	DESCRIPCIÓN
`literal`	Representa un elemento que admite una serie de valores concretos
`enum`	Representa un conjunto de constantes con nombre
`class`	Representa una plantilla para crear objetos con propiedades y métodos definidos
`array`	Representa una lista de valores
`tuple`	Representa una colección con un número fijo de elementos
`function`	Representa una referencia función

Las composiciones de tipos:

TIPO	DESCRIPCIÓN
`union`	Representa un valor que puede ser uno de varios tipos especificados
`intersection`	Representa un valor que debe cumplir con todos los tipos especificados

8. Tipos de datos en TypeScript

Los tipos **definidos por el usuario** (o personalizados), nos permiten crear nuevos tipos a partir de la composición de otros tipos.

Estos tipos pueden representar estructuras de datos complejas, uniones de tipos múltiples, o incluso comportamientos.

EJEMPLOS DE TÌPOS DE DATOS PERSONALIZADOS

1) **literal** - permiten definir un conjunto de valores específicos que una variable puede tomar.

```
let direction: "up" | "down" | "left" | "right";
direction = "up";
```

2) **enum** - permite definir un conjunto de constantes con nombres más descriptivos.

```
enum Direction {
  Up = 1,
  Down,
  Left,
  Right
}
let userDirection: Direction = Direction.Up;
```

8. Tipos de datos en TypeScript

3) **Tuplas** - son una forma de definir un array con un número fijo de elementos con tipos de datos específicos.

```
let employee: [string, number];
employee = ["John Doe", 35];
```

EJEMPLOS DE TIPOS DE DATOS ORIENTADOS A OBJETOS

1) clases - son una forma de definir objetos con métodos y propiedades. Permiten crear instancias de objetos y heredar propiedades y métodos de otras clases.

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
  makeSound() {
    console.log("Animal is making a sound");
  }
}
```

8. Tipos de datos en TypeScript

2) interface - son una forma de definir la estructura de un objeto en Typescript. Pueden contener propiedades, métodos y eventos.

```
interface Person {
  name: string;
  age: number;
  greet(): void;
}
```

TIPOS COMPUESTOS

1) union - permite definir una variable que puede aceptar varios tipos de datos diferentes.

```
let age: number | string;
age = 25;
age = "25";
```

2) Intersección - permite combinar dos o más tipos de datos en uno solo.

```
type Admin = {
  name: string;
  isAdmin: boolean;
}

type Employee = {
  name: string;
  salary: number;
}

type AdminEmployee = Admin & Employee;
```

8. Tipos de datos en TypeScript

ALIAS

Un **Alias** de tipo en TypeScript nos permite crear un nombre alternativo para un tipo específico (ya sea estándar, o definido por nosotros).

Los alias son una gran ayuda para mejorar la legibilidad del código. Son especialmente útiles cuando trabajamos con tipos complejos (como uniones, intersecciones y tipos personalizados)

CREACIÓN DE UN ALIAS

Para crear un alias de tipo se utiliza la palabra clave **type** seguida del nombre del alias y la definición del tipo.

```
type NombreAlias = Tipo;
```

Ejemplo:

```
type ID = number;         // ahora ID es un alias de number
type Nombre = string;     // Nombre es un alias de string

let usuarioID: ID = 123;
let usuarioNombre: Nombre = "Luis Pérez";

console.log(usuarioID); // 123
console.log(usuarioNombre); // Luis Pérez
```

- ID es un alias para el tipo number
- Nombre es un alias para el tipo string

8. Tipos de datos en TypeScript

ALIAS CON TIPOS COMPUESTOS

Los alias de tipo son especialmente útiles cuando se combinan tipos mediante uniones o intersecciones.

ALIAS CON UNIONES

```
// Definimos un alias de tipo 'Resultado' que puede ser una cadena de texto o un número
type Resultado = string | number;

// Creamos una variable 'valor1' con un texto, y valor2 con un numero
let valor1: Resultado = "Éxito";
let valor2: Resultado = 200;

console.log(valor1); // Éxito
console.log(valor2); // 200
```

Resultado es un tipo que puede ser o una cadena de texto (string) o un número (number).

8. Tipos de datos en TypeScript

ALIAS CON INTERSECCIONES

```
// Definimos una interfaz 'Usuario' con propiedades 'nombre' y 'email'
interface Usuario {
    nombre: string;
    email: string;
}

// Definimos una interfaz 'Empleado' con propiedades 'empleadoID' y 'departamento'
interface Empleado {
    empleadoID: number;
    departamento: string;
}

// Creamos un alias de tipo 'UsuarioEmpleado' que combina las interfaces 'Usuario' y 'Empleado'
type UsuarioEmpleado = Usuario & Empleado;

// Creamos un objeto 'empleado' que debe cumplir con ambas interfaces: 'Usuario' y 'Empleado'
const empleado: UsuarioEmpleado = {
    nombre: "Ana García",
    email: "ana.garcia@example.com",
    empleadoID: 101,
    departamento: "Desarrollo"
};

console.log(empleado.nombre); // Ana García
console.log(empleado.departamento); // Desarrollo
```

UsuarioEmpleado es un tipo que combina dos interfaces: Usuario y Empleado. Esto significa que cualquier objeto de tipo UsuarioEmpleado debe tener todas las propiedades definidas en ambas interfaces.

8. Tipos de datos en TypeScript

ALIAS CON LITERALES

```
// Definimos un tipo literal 'Estado' que puede ser solo uno de los valores específicos
type Estado = 'activo' | 'inactivo' | 'pendiente';

// Creamos una variable 'estadoUsuario' del tipo 'Estado'
let estadoUsuario: Estado = 'activo';
console.log(estadoUsuario); // activo

// Cambiamos el valor de 'estadoUsuario' a otro valor válido
estadoUsuario = 'pendiente';
console.log(estadoUsuario); // pendiente

// Intentar asignar un valor no permitido causará un error de compilación
estadoUsuario = 'suspendido'; // Error: Type '"suspendido"' is not assignable to type 'Estado'.
```

ALIAS CON TIPOS DE FUNCIONES

```
type Operacion = (a: number, b: number) => number;

const sumar: Operacion = (a, b) => a + b;
const restar: Operacion = (a, b) => a - b;

console.log(sumar(5, 3)); // 8
console.log(restar(5, 3)); // 2
```

Operacion es un alias para cualquier función que tome dos números como parámetros y retorne un número.

8. Tipos de datos en TypeScript

ALIAS CON TIPOS GENÉRICOS

```
type Respuesta<T> = {
    exito: boolean;
    datos: T;
    mensaje?: string;
};

const respuesta1: Respuesta<string> = {
    exito: true,
    datos: "Operación exitosa"
};

const respuesta2: Respuesta<number> = {
    exito: false,
    datos: 0,
    mensaje: "Error en la operación"
};

console.log(respuesta1.datos); // Operación exitosa
console.log(respuesta2.mensaje); // Error en la operación
```

Respuesta es un alias de tipo genérico que puede adaptarse a diferentes tipos de datos.

8. Tipos de datos en TypeScript

ALIAS Y TIPOS AVANZADOS

ALIAS Y TIPOS CONDICIONALES

Los alias pueden combinarse con tipos condicionales para crear tipos más dinámicos y adaptativos.

```
type TipoTexto<T> = T extends string ? "Es una cadena" : "No es una cadena";

type ResultadoTexto = TipoTexto<string>; // Es una cadena
type ResultadoNumero = TipoTexto<number>; // No es una cadena
```

ALIAS Y TIPOS DE MAPEO

Los alias pueden usarse con tipos de mapeo para transformar otros tipos.

```
type Opcional<T> = {
    [P in keyof T]?: T[P];
};

interface Usuario {
    nombre: string;
    email: string;
}

type UsuarioOpcional = Opcional<Usuario>;

const usuario: UsuarioOpcional = {
    nombre: "Carlos"
};

console.log(usuario.nombre); // Carlos
console.log(usuario.email); // undefined
```

TIPOS GENÉRICOS

Los tipos genéricos permiten escribir funciones y clases que pueden trabajar con cualquier tipo de datos. Por ejemplo:

```
function identidad<T>(arg: T): T {
    return arg;
}
```

T es un tipo genérico que se determina en tiempo de ejecución.

Los genéricos en TypeScript nos permiten definir funciones, clases o componentes que admiten un tipo como parte de su definición.

De esta forma, pueden comportarse de forma diferente, en función del tipo recibido, adaptando su comportamiento genérico a los diferentes tipos.

DEFINICIÓN DE FUNCIONES GÉNERICAS

Los genéricos se definen utilizando el símbolo < >, y se pueden aplicar tanto a variables como a funciones.

8. Tipos de datos en TypeScript

Ejemplo función genérica que acepta cualquier tipo de dato:

```
function imprimirValor<T>(valor: T): void {
    console.log(valor);
}
```

- <T> es el tipo genérico que se puede utilizar para representar cualquier tipo de dato.
- La función imprimirValor acepta un parámetro valor de tipo T y simplemente lo imprime en la consola.

CREACIÓN DE UNA CLASE GÉNERICA - clases que pueden trabajar con diferentes tipos de datos.

```
class Coleccion<T> {
    private elementos: T[] = [];

    agregar(elemento: T): void {
        this.elementos.push(elemento);
    }

    obtener(indice: number): T {
        return this.elementos[indice];
    }
}
```

- Hemos creado una clase genérica llamada Coleccion que puede almacenar cualquier tipo de dato.
- La clase tiene un método agregar que acepta un elemento de tipo T y lo añade a una lista interna.
- También tiene un método obtener que devuelve el elemento en un índice específico.

RESTRICCIONES DE TIPOS GENÉRICOS

En ocasiones, es posible que queramos restringir los tipos que se pueden utilizar con un genérico (por ejemplo, si queremos asegurarnos de que solo se puedan utilizar tipos numéricos con una función genérica)

Podemos lograr esto utilizando restricciones de tipo en los genéricos.

```
function sumarNumeros<T extends number>(a: T, b: T): number {
    return a + b;
}
```

- Hemos utilizado la palabra clave **extends** para indicar que el tipo genérico T debe ser un subtipo de number.
- Esto nos permite utilizar la función sumarNumeros con cualquier tipo numérico, pero no con otros tipos de datos.

INFERENCIA DE TIPOS GENÉRICOS

TypeScript tiene la capacidad de inferir automáticamente el tipo genérico en función del tipo de datos proporcionado, es deicr, que no tenemos que ponerlo nosotros.

Esto nos permite escribir código más limpio y conciso sin tener que especificar explícitamente el tipo genérico en cada llamada.

```
function imprimirValor<T>(valor: T): void {
    console.log(valor);
}
```

Si llamamos a esta función sin especificar el tipo genérico:

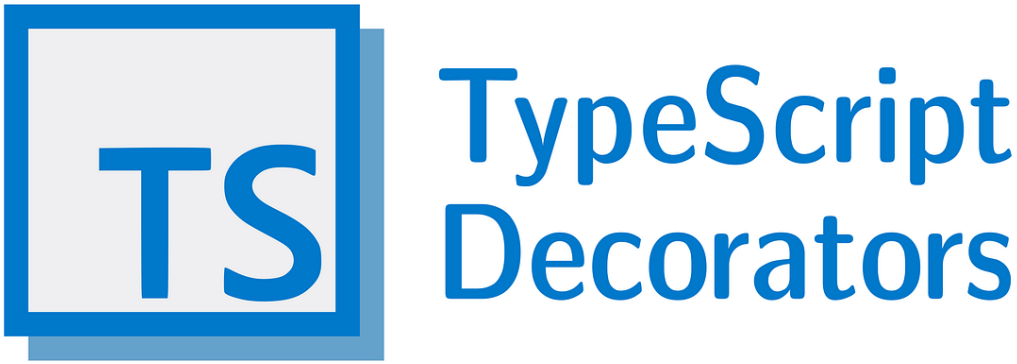
```
imprimirValor(42);
```

TypeScript inferirá automáticamente que el tipo genérico es number, ya que hemos pasado un número como argumento.

DECORADORES

Los decoradores se utilizan para añadir metadatos a las clases y sus miembros. Por ejemplo, un decorador de clase:

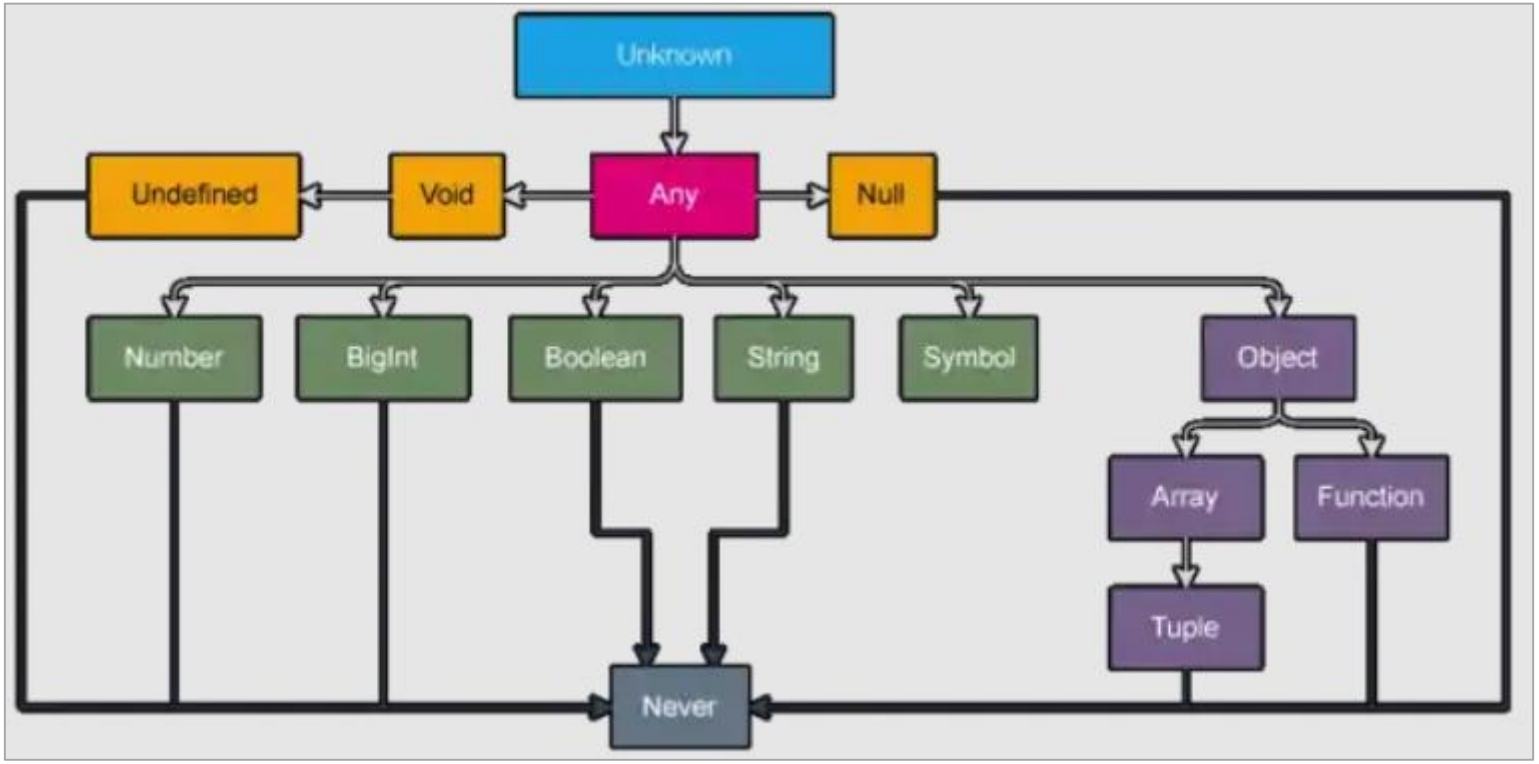
```
function logClase(target: Function) {
    console.log(`Clase: ${target.name}`);
}

@logClase
class MiClase {
    // Implementación de la clase
}
```



8. Tipos de datos en TypeScript

JERARQUÍA DE TIPOS

La jerarquía de tipos en TypeScript se organiza de manera que cada tipo se deriva o es compatible con otros tipos en el sistema.



9. Tipo Number en TypeScript

En TypeScript, el tipo **number** es utilizado para representar números, tanto enteros como de punto flotante.

```
let entero: number = 42;
let decimal: number = 3.14;
let hex: number = 0xff;    // Notación hexadecimal
let bin: number = 0b1010;  // Notación binaria
let oct: number = 0o744;   // Notación octal
```

OPERACIONES ARITMÉTICAS

TypeScript permite realizar las operaciones aritméticas básicas con el tipo number: suma, resta, multiplicación, división y módulo.

```
let a: number = 10;
let b: number = 3;

console.log(a + b);  // 13
console.log(a - b);  // 7
console.log(a * b);  // 30
console.log(a / b);  // 3.3333333333333335
console.log(a % b);  // 1
```

9. Tipo Number en TypeScript

TypeScript proporciona varios métodos y propiedades estáticas en el **objeto Number** para ayudarnos a trabajar con números:

- **Number.MAX_VALUE:** El valor numérico máximo representable.
- **Number.MIN_VALUE:** El valor numérico positivo más pequeño representable.
- **Number.POSITIVE_INFINITY:** Representa el infinito positivo.
- **Number.NEGATIVE_INFINITY:** Representa el infinito negativo.
- **Number.NaN:** Representa un valor que no es un número (Not-A-Number).

```
console.log(Number.MAX_VALUE);  // 1.7976931348623157e+308
console.log(Number.MIN_VALUE);  // 5e-324
console.log(Number.POSITIVE_INFINITY);  // Infinity
console.log(Number.NEGATIVE_INFINITY);  // -Infinity
console.log(Number.NaN);  // NaN
```

9. Tipo Number en TypeScript

CONVERSIÓN DE TIPOS

TypeScript proporciona varias formas de texto a números, incluyendo **parseInt**, **parseFloat** y el operador unario **+**.

```
let cadena: string = "42";
let entero: number = parseInt(cadena);  // 42
let flotante: number = parseFloat("3.14");  // 3.14
let numero: number = +cadena;  // 42
```

CONVERSIÓN DE NÚMERO A STRING

Para convertir un número a una cadena, se puede utilizar el método **toString**.

```
let num: number = 42;
let str: string = num.toString();  // "42"
```

NÚMEROS ESPECIALES

NaN es un valor especial que representa un valor no numérico. Es el resultado de operaciones no definidas o ilegales, como dividir 0 por 0.

```
let nan: number = 0 / 0;
console.log(nan);  // NaN
console.log(isNaN(nan));  // true
```

10. Tipo String en TypeScript

En TypeScript, el tipo **string** se utiliza para representar secuencias de caracteres. Las cadenas de caracteres pueden ser definidas utilizando comillas simples ('), comillas dobles ("), o backticks (`) para plantillas de cadenas (template strings).

```
let mensaje: string = "Hola, TypeScript!";
let saludo: string = '¡Bienvenido!';
let plantilla: string = `Este es un mensaje de TypeScript`;
```

TEMPLATE STRINGS

Las plantillas de cadenas (template strings) permiten incluir expresiones embebidas, lo que facilita la construcción de cadenas complejas. Se delimitan con backticks (`) y pueden contener expresiones interpoladas usando ${}.

```
let nombre: string = "Luis";
let edad: number = 30;
let mensaje: string = `Hola, mi nombre es ${nombre} y tengo ${edad} años.`;
console.log(mensaje);  // "Hola, mi nombre es Luis y tengo 30 años."
```

10. Tipo String en TypeScript

CARACTERES DE ESCAPE

Para incluir caracteres especiales en una cadena, se utilizan secuencias de escape.

- **\n:** Nueva línea
- **\t:** Tabulación
- **\\:** Barra invertida
- **\':** Comilla simple
- **\":** Comilla doble

```
let multilinea: string = "Línea 1\nLínea 2";
console.log(multilinea);
// Línea 1
// Línea 2
```

CONCATENACIÓN DE CADENAS

La concatenación de cadenas se puede realizar usando el operador **+** o los métodos **concat** y **template strings**.

```
let saludo: string = "Hola, " + nombre + ". Bienvenido a " + texto + "!";
console.log(saludo);  // "Hola, Luis. Bienvenido a TypeScript!"
```

```
let saludoConcatenado: string = saludo.concat(" ¡Disfruta tu aprendizaje!");
console.log(saludoConcatenado);  // "Hola, Luis. Bienvenido a TypeScript! ¡Disfru
```

```
let saludoTemplate: string = `Hola, ${nombre}. Bienvenido a ${texto}!`;
console.log(saludoTemplate);  // "Hola, Luis. Bienvenido a TypeScript!"
```

10. Tipo String en TypeScript

COMPARACIÓN DE CADENAS

Las cadenas se pueden comparar usando los operadores de comparación (<, >, <=, >=, ==, !=, ===, !==). Las comparaciones son sensibles a mayúsculas y minúsculas.

```
let cadena1: string = "abc";
let cadena2: string = "Abc";

console.log(cadena1 === cadena2);  // false
console.log(cadena1.toLowerCase() === cadena2.toLowerCase());  // true
```

MÉTODOS Y PROPIEDADES DE STRING

TypeScript proporciona una variedad de métodos y propiedades para manipular cadenas de caracteres. Los más utilizados son:

1) Length
2) charAt(index)
3) Concat(…cadenas)
4) Includes(subcadena)
5) IndexOf (subcadena)
6) replace(subcadena, nuevaCadena)
7) Split(separador)
8) substring(inicio, fin)
9) toLowerCase()
10) toUpperCase()

String
https://www.tutorialsteacher.com/typescript/typescript-string

11. Tipo Boolean en TypeScript

En TypeScript, el tipo boolean representa uno de dos valores: true o false.

Estos valores se utilizan comúnmente para controlar el flujo del programa, realizar comparaciones y expresar condiciones.

La declaración de variables booleanas en TypeScript se realiza especificando el tipo **boolean** y asignando un valor **true** o **false**.

```
let esActivo: boolean = true;
let estaAutenticado: boolean = false;
```

OPERADORES BOOLEANOS

Los operadores de comparación devuelven un valor booleano como resultado de la comparación entre dos valores.

- == (igualdad) y === (igualdad estricta)
- != (desigualdad) y !== (desigualdad estricta)
- <, <=, >, >=

```
let x: number = 10;
let y: number = 20;

console.log(x == y);  // false
console.log(x != y);  // true
console.log(x < y);   // true
console.log(x >= y);  // false
```

OPERADORES LÓGICOS

&& (AND lógico) – devuelve true si ambos operandos son true

```
let a: boolean = true;
let b: boolean = false;
console.log(a && b);  // false
```

! (NOT lógico) – Invierte el valor booleano

```
console.log(!a);  // false
console.log(!b);  // true
```

|| (OR lógico) - Devuelve true si al menos uno de los operandos es true

```
console.log(a || b);  // true
```

10. Tipo String en TypeScript

CONVERSIÓN DE TIPOS

DE VALORES A BOOLEANOS – JavaScript (y por extensión TypeScript) tiene conceptos de valores **truthy** y **falsy**.

Estos valores se convierten implícitamente a booleanos en contextos booleanos.

Los valores falsy comunes son:

- `false`
- `0`
- `""` (cadena vacía)
- `null`
- `undefined`
- `NaN`

El resto de valores son **truthy**

```
console.log(Boolean(0));          // false
console.log(Boolean(""));         // false
console.log(Boolean("Hello"));    // true
console.log(Boolean(123));        // true
```

10. Tipo String en TypeScript

DE BOOLEANOS A VALORES – Los booleanos se pueden convertir a otros tipos de datos, aunque no es una práctica común.

```
let verdadero: boolean = true;
let falso: boolean = false;

console.log(Number(verdadero));  // 1
console.log(Number(falso));      // 0
console.log(String(verdadero));  // "true"
console.log(String(falso));      // "false"
```

11. Tipos especiales

ANY – es un tipo especial que permite almacenar valores de cualquier tipo.

Utilizar **any** es similar a trabajar con JavaScript puro, donde no se imponen restricciones de tipo en las variables.

Una variable de tipo **any** puede contener valores de cualquier tipo y puede cambiar su tipo en tiempo de ejecución.

```
let variable: any;

variable = 42;          // number
variable = "cadena";    // string
variable = true;        // boolean
variable = { a: 1 };    // object
variable = [1, 2, 3];   // array
```

Cuando una variable o parámetro se declara con el tipo **any**, TypeScript no realiza ninguna verificación de tipo en tiempo de compilación para esa variable.

Una variable **any** puede contener cualquier tipo de valor, y se pueden realizar operaciones sobre ella sin que el compilador genere errores de tipo.

Esto proporciona una gran flexibilidad, pero a costa de perder la seguridad de tipos que ofrece TypeScript.

USOS COMUNES DE ANY

MIGRACIÓN DESDE JAVASCRIPT

Uno de los usos más comunes de any es durante la migración de un código de JavaScript a TypeScript. Al utilizar any, se puede ir introduciendo el tipado de manera gradual.

```
function procesarDatos(datos: any): void {
    console.log(datos);
}
```

DATOS DE ORIGEN DESCONOCIDO

Cuando se trabaja con datos cuyo tipo no se conoce de antemano (como datos provenientes de una API externa) se puede utilizar any.

```
async function obtenerDatos(url: string): Promise<any> {
    let response = await fetch(url);
    return response.json();
}
```

11. Tipos especiales

IMPLEMENTACIÓN DE LIBRERÍAS GENÉRICAS

En algunas librerías genéricas o de utilidad, el tipo any puede ser útil o necesario para trabajar con ellas.

```
function clonar<T>(objeto: T): T {
    return JSON.parse(JSON.stringify(objeto));
}

let copia: any = clonar({ a: 1, b: "texto" });
```

BUENAS PRÁCTICAS

1) **MINIMIZAR EL USO DE ANY** - Utilizar **any** solo cuando sea absolutamente necesario. Siempre que sea posible, utiliza tipos más específicos para aprovechar las ventajas de TypeScript.
2) **CONVERSIÓN DE TIPOS (TYPE ASSERTIONS)** - Cuando se tiene certeza del tipo de una variable, se puede utilizar la conversión de tipos en lugar de any.
3) **TIPOS PERSONALIZADOS Y GENÉRICOS** - Siempre que sea posible, define tipos personalizados o utiliza genéricos para mantener la seguridad de tipos.

11. Tipos especiales

USO DE NULL Y UNDEFINED

En algunas librerías genéricas o de utilidad, el tipo any puede ser útil o necesario para trabajar con ellas.

De forma resumida:

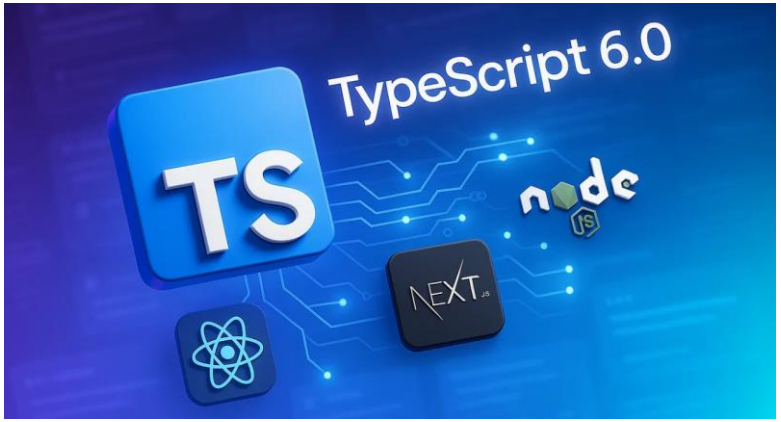
Undefined: Es un valor que indica que una variable ha sido declarada pero no inicializada.

Null: Es un valor que se asigna explícitamente a una variable para indicar la ausencia intencional de un valor.

CONFIGURACIÓN STRICT NULL CHEKS

Cuando **strictNullChecks** está habilitado en TypeScript, null y undefined no son subtipos de todos los demás tipos.

Esto significa que una variable de tipo string no puede contener null o undefined a menos que se especifique explícitamente.



11. Tipos especiales

VOID

El tipo **void** en TypeScript representa la ausencia de un valor. Es similar al concepto de void que encontramos en otros lenguajes de programación (como C++ y C#).

Se utiliza principalmente en el contexto de funciones para indicar que una función no retorna ningún valor significativo.

Cuando una función está diseñada para realizar una acción (a veces llamado efecto secundario), como modificar un estado o imprimir un mensaje en la consola. Entonces no necesita devolver un resultado, se especifica el tipo void.

NEVER

El tipo **never** en TypeScript representa el conjunto de valores para las funciones que nunca retornan un valor válido.

Esto generalmente significa que la función arroja una excepción o que entra en un bucle infinito y nunca completa su ejecución de manera normal.

El tipo **never** se usa principalmente en dos contextos:

- Funciones que lanzan excepciones o entran en bucles infinito
- En las estructuras de control de flujo para manejar casos inalcanzables.

EJERCICIO 1

Transforma estas funciones Javascript sin tipo en funciones TypeScript:

```
// Versión sin tipos (JS)
function logMensaje(msg) {
  console.log(msg);
}
```

```
// Versión con tipos en TypeScript
function logMensajeTS(msg: string): void {
  console.log(msg);
}
```

```
function formatCurrency(value) {
  return '$' + value.toFixed(2);
}
```

```
function formatCurrency(value: number): string {
  return '$' + value.toFixed(2);
}

// Uso con inferencia de tipos
const precio = 19.99;
console.log(formatCurrency(precio)); // '$19.99'
```

EJERCICIO 2

Transforma estas funciones Javascript sin tipo en funciones TypeScript:

```
var array = [];
array.push({ 'name': 'hola', 'list': [] });
array[0]['list'].push('list1');
alert(array[0]['name']);
```

```
interface Datos {
  name: string;
  list: string[];
}

const array: Datos[] = [];
array.push({ name: 'hola', list: [] });
array[0].list.push('list1');
alert(array[0].name);
```

```
function validarUsuario(usuario) {
  return usuario.length >= 3;
}
```

```
interface UsuarioForm {
  nombre: string;
  email: string;
  edad: number;
}

function validarUsuario(usuario: UsuarioForm): boolean {
  return (
    usuario.nombre.length >= 3 &&
    usuario.email.includes('@') &&
    usuario.edad >= 18
  );
}
```

EJERCICIO 3

Toma estas funciones JS y conviértelas a TS:

- function suma(a, b) { return a + b; }
- function filtrarPares(numeros) { return numeros.filter(n => n % 2 === 0); }
- function crearPersona(nombre, edad) { return { nombre, edad }; }

SOLUCIÓN:

```
// 1
function suma(a: number, b: number): number { return a + b; }

// 2
function filtrarPares(numeros: number[]): number[] {
  return numeros.filter(n => n % 2 === 0);
}

// 3
interface Persona { nombre: string; edad: number; }
function crearPersona(nombre: string, edad: number): Persona {
  return { nombre, edad };
}
```

12. Tipos personalizados

LITERALES EN TYPESCRIPT

LITERALES - tipos que permiten definir variables con valores específicos y prefijados en lugar de tipos amplios como string o number.

Permiten un mayor control y seguridad sobre los valores que puede tomar una variable, limitando las opciones disponibles.

Tipos:

1) **LITERALES DE CADENA** - permiten definir una variable que solo puede tener uno de varios valores específicos de cadena.

```
type Direccion = "norte" | "sur" | "este" | "oeste";

let direccion: Direccion;

direccion = "norte"; // Correcto
direccion = "sur";   // Correcto
```

2) **LITERALES NUMÉRICOS** - permiten que una variable tenga solo ciertos valores numéricos específicos.

```
type UnoADiez = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10;
```

12. Tipos personalizados

3) LITERALES Y TIPOS UNIÓN - los literales se utilizan frecuentemente junto con los tipos unión para crear variables que pueden aceptar varios valores específicos.

```
type Respuesta = "sí" | "no" | "tal vez";

function contestar(respuesta: Respuesta): void {
    console.log(`La respuesta es: ${respuesta}`);
}

contestar("sí");      // Correcto
contestar("no");      // Correcto
contestar("tal vez"); // Correcto
```

4) LITERALES Y TIPOS PERSONALIZADOS - los literales también pueden ser utilizados en la definición de tipos personalizados más complejos, proporcionando mayor control sobre los valores de las propiedades de los objetos.

```
type Estado = "iniciado" | "en progreso" | "completado";

interface Tarea {
    titulo: string;
    descripcion: string;
    estado: Estado;
}

let tarea: Tarea = {
    titulo: "Aprender TypeScript",
    descripcion: "Estudiar los tipos literales en TypeScript.",
    estado: "en progreso"
};
```

ARRAY

ARRAY - colección ordenada de elementos, que pueden ser de cualquier tipo (aunque generalmente son homogéneos).

Existen dos sintaxis principales para declarar un array en TypeScript:

1) La más habitual es empleando corchetes ([])

```
let numeros: number[] = [1, 2, 3, 4, 5];
let palabras: string[] = ["TypeScript", "JavaScript", "Node.js"];
```

2) Usando la Clase Genérica Array<T>

```
let numeros: Array<number> = [1, 2, 3, 4, 5];
let palabras: Array<string> = ["TypeScript", "JavaScript", "Node.js"];
```

Los **arrays multidimensionales** (**matrices**) se pueden declarar anidando arrays.

```
let matriz: number[][] = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];
```

ITERACCIÓN SOBRE ARRAYS

TypeScript soporta la iteración mediante bucles tradicionales y la sintaxis **for...of**

```
for (let i = 0; i < numeros.length; i++) {
    console.log(numeros[i]);
}

for (let num of numeros) {
    console.log(num);
}
```

También se puede usar **forEach**

```
numeros.forEach((num) => console.log(num));
```

TypeScript hereda los métodos y propiedades de arrays de JavaScript, por lo que tenemos una gran cantidad de funciones disponibles manipular y operar con arrays.

Arrays
https://es.javascript.info/array-methods

EJERCICIO 4

Dado el array 1, 2, 3, 4, 5, 2, 3, 4, crear un programa con TypeScript que encuentre los elementos duplicados:

SOLUCIÓN:

```
1  function encontrarDuplicados(arr: number[]): number[] {
2      let duplicados: number[] = [];
3      let contador: { [key: number]: number } = {};
4
5      arr.forEach((num) => {
6          contador[num] = (contador[num] || 0) + 1;
7      });
8
9      for (let num in contador) {
10         if (contador[num] > 1) {
11             duplicados.push(Number(num));
12         }
13     }
14
15     return duplicados;
16 }
17
18 let numeros: number[] = [1, 2, 3, 4, 5, 2, 3, 4];
19 console.log(encontrarDuplicados(numeros));  // [2, 3, 4]
```

EJERCICIO 5

Dado un array de objetos con estos datos. Ordenarlo de forma creciente:

{ nombre: "Luis", edad: 25 }		{ nombre: "Ana", edad: 22 },
{ nombre: "Ana", edad: 22 }	→	{ nombre: "Luis", edad: 25 }
{ nombre: "Luis", edad: 30 }		{ nombre: "Luis", edad: 30 }

SOLUCIÓN:

```typescript
interface Persona {
    nombre: string;
    edad: number;
}

let personas: Persona[] = [
    { nombre: "Luis", edad: 25 },
    { nombre: "Ana", edad: 22 },
    { nombre: "Luis", edad: 30 }
];

personas.sort((a, b) => a.edad - b.edad);
console.log(personas);
```

12. Tipos personalizados

ENUM

ENUMERACIÓN - característica de TypeScript que permite definir un conjunto de valores constantes bajo un mismo nombre.

Para definir una enumeración, se utiliza la palabra clave **enum** seguida del nombre y un bloque de llaves que contiene los valores.

```
enum Direccion {
    Norte,
    Sur,
    Este,
    Oeste
}

let direccionActual: Direccion = Direccion.Norte;

console.log(direccionActual); // Output: 0
console.log(Direccion[direccionActual]); // Output: Norte
```

Direccion es un enum que define cuatro valores: Norte, Sur, Este y Oeste.

Los valores del enum son automáticamente asignados a números enteros, comenzando desde 0.

12. Tipos personalizados

Por defecto, los **enums** en TypeScript son numéricos.

Cada valor en el enum se asigna a un número, comenzando desde 0 y aumentando en 1 para cada valor subsiguiente.

Se pueden **personalizar los valores** numéricos asignados a cada miembro del enum.

Si se define un valor numérico explícito para un miembro del enum, los valores subsiguientes se incrementarán automáticamente a partir de ese valor.

```
enum Estado {
    Inactivo = 0,
    Activo = 1,
    Pausado = 2
}

let estadoActual: Estado = Estado.Activo;

console.log(estadoActual); // Output: 1
```

```
enum Semana {
    Lunes = 1,
    Martes,
    Miercoles,
    Jueves,
    Viernes
}

console.log(Semana.Lunes); // Output: 1
console.log(Semana.Martes); // Output: 2
console.log(Semana.Viernes); // Output: 5
```

12. Tipos personalizados

TypeScript también permite definir enums de string, donde cada miembro del enum se asigna a una cadena de texto.

```
enum Color {
    Rojo = "Rojo",
    Verde = "Verde",
    Azul = "Azul"
}

let colorFavorito: Color = Color.Verde;

console.log(colorFavorito); // Output: Verde
```

Aunque no es una práctica común, ni recomendable, TypeScript permite definir enums que mezclan valores numéricos y de cadena.

```
enum Mixto {
    No = 0,
    Si = "Sí"
}

console.log(Mixto.No); // Output: 0
console.log(Mixto.Si); // Output: Sí
```

TUPLAS

TUPLA - tipo especial de array que permite almacenar un número fijo de elementos con tipos específicos en cada posición.

Las tuplas son útiles cuando sabemos exactamente cuántos y qué tipos de elementos queremos almacenar en una agrupación.

Para declarar una tupla, se utiliza una sintaxis similar a la de los arrays, pero especificando los tipos de cada elemento entre corchetes.

```
let tupla: [number, string, boolean];
tupla = [1, "TypeScript", true];
```

tupla puede contener un número, una cadena y un valor booleano, en ese orden específico.

Las tuplas deben ser inicializadas con el número y tipo exacto de elementos especificados en su declaración.

Intentar asignar un número diferente de elementos (más o menos) o elementos de tipos incorrectos causará un error de compilación.

ACCESO Y MANIPULACIÓN DE TUPLAS

Los elementos de una tupla se pueden acceder utilizando el índice (igual que en un array).

```
let miTupla: [string, number, boolean] = ["TypeScript", 2024, true];

console.log(miTupla[0]);  // "TypeScript"
console.log(miTupla[1]);  // 2024
console.log(miTupla[2]);  // true
```

Se pueden reasignar los valores de los elementos de una tupla utilizando su índice, pero deben mantener el tipo original.

```
miTupla[1] = 2025;  // Correcto
miTupla[2] = false;  // Correcto

// Incorrecto
// miTupla[1] = "2025";  // Error: Tipo incorrecto
```

TypeScript soporta la desestructuración de tuplas lo que permite asignar los valores de una tupla a variables individuales.

```
let [lenguaje, año, activo] = miTupla;
console.log(lenguaje);  // "TypeScript"
console.log(año);       // 2024
console.log(activo);    // true
```

MÉTODOS EN TUPLAS

Las tuplas heredan varios métodos de arrays, aunque su uso puede ser limitado debido a la naturaleza fija de las tuplas.

1) **push()** - Añade un elemento al final de la tupla. Sin embargo, esto puede romper la tupla si se agregan elementos de tipos no esperados.

```
miTupla.push("nuevo elemento");
```

2) **pop()** - Elimina el último elemento de la tupla.

```
let ultimoElemento = miTupla.pop();
```

3) **concat()** - Combina dos o más tuplas

```
let miTupla: [string, number, boolean] = ["TypeScript", 2024, true];
let otraTupla: [string, number] = ["Hola", 100];
let combinada = miTupla.concat(otraTupla);

console.log(combinada);  // ["TypeScript", 2024, false, "Hola", 100]
```

TIPOS AVANZADOS DE TUPLAS

TypeScript permite definir tuplas con elementos opcionales utilizando el símbolo **?**

```
let tuplaOpcional: [number, string?, boolean?];
tuplaOpcional = [1];
tuplaOpcional = [1, "Opcional"];
tuplaOpcional = [1, "Opcional", true];
```

Las tuplas también pueden utilizar rest elements (**...**) para permitir un número variable de elementos de un tipo específico al final de la tupla.

```
let tuplaRest: [number, ...string[]];
tuplaRest = [1];
tuplaRest = [1, "a", "b", "c"];
```

Las tuplas pueden ser muy útiles como parámetros de funciones para asegurar que se reciben un conjunto específico de tipos y en un orden particular.

```
function procesarTupla(tupla: [string, number, boolean]): void {
    let [texto, numero, booleano] = tupla;
    console.log(`Texto: ${texto}, Número: ${numero}, Booleano: ${booleano}`);
}

procesarTupla(["TypeScript", 2024, true]);
```

UNIÓN

Los tipos **unión** en TypeScript permiten que una variable o una función acepten múltiples tipos de datos.

Esto nos da flexibilidad para definir tipos y nos ayuda en situaciones donde los valores pueden ser de más de un tipo.

Para declarar un tipo unión, se utiliza el operador de barra vertical (|) entre los tipos que se quieren combinar. Esto indica que una variable puede ser de uno u otro tipo.

```
let valor: number | string;

valor = 42;        // Correcto
valor = "texto";   // Correcto

valor = true;      // ❌ Error: El tipo 'boolean' no se puede asignar al tipo 'number
```

La variable **valor** puede contener un número o una cadena de texto, pero no un valor booleano.

INTERSECCIÓN

Las **intersecciones** en TypeScript nos permiten combinar múltiples tipos en un solo tipo, es decir, sirven para crear un tipo que suma las propiedades de dos tipos.

El **Intersection Type** se define con el operador **&,** el tipo resultante contiene todas las propiedades de los tipos combinados.

```
type A = { a: number };
type B = { b: string };

type C = A & B;
// C es equivalente a { a: number; b: string }
```

El **tipo C** es una intersección de los tipos A y B

Esto significa que C tendrá tanto la propiedad a del tipo A como la propiedad b del tipo B.

13. Tipos compuestos

Una de las formas más comunes de usar tipos de intersección es **combinando interfaces**.

Esto es especialmente útil cuando queremos crear un objeto que tenga propiedades de varias interfaces:

```
interface Identificable {
    id: number; // Propiedad id de tipo number
}

interface Nombrable {
    nombre: string; // Propiedad nombre de tipo string
}

// Definimos un tipo Persona que es la intersección de Identificable y Nombrable
type Persona = Identificable & Nombrable;

const persona: Persona = {
    id: 1,
    nombre: "Luis"
};

console.log(persona.id); // Imprime: 1
console.log(persona.nombre); // Imprime: Luis
```

Persona es un tipo de intersección que combina las propiedades de Identificable y Nombrable, por lo tanto, **persona** debe tener tanto un id como un nombre

13. Tipos compuestos

Las intersecciones también son útiles para extender tipos existentes, por ejemplo, si se quiere añadir propiedades adicionales a un tipo ya definido.

```
interface Base {
  id: number;
}

type ConNombre = Base & { nombre: string };

const item: ConNombre = {
  id: 1,
  nombre: "Item 1"
};
```

ConNombre extiende el tipo Base añadiendo una propiedad nombre.

Las intersecciones pueden combinarse con tipos genéricos para crear tipos más flexibles y reutilizables.

La función **combinar** acepta dos objetos de tipos genéricos T y U

Retorna un objeto que es la intersección de ambos tipos.

```
function combinar<T, U>(obj1: T, obj2: U): T & U {
    return { ...obj1, ...obj2 };
}

const objeto1 = { nombre: "Luis" };
const objeto2 = { edad: 30 };

const objetoCombinado = combinar(objeto1, objeto2);

console.log(objetoCombinado.nombre); // Luis
console.log(objetoCombinado.edad); // 30
```

14. Funciones y métodos

TypeScript permite una gran variedad de tipos de **parámetros** en sus funciones. Tipos de parámetros

1) Obligatorios
2) Opcionales
3) Por defecto
4) Rest
5) Con tipos literales
6) De función con tipo

1.- PARÁMETROS OBLIGATORIOS

PARÁMETROS OBLIGATORIOS - son aquellos que deben ser proporcionados cuando se llama a la función. Estos parámetros se definen en la declaración de la función sin ningún modificador especial.

```
function saludar(nombre: string, edad: number): void {
    console.log(`Hola ${nombre}, tienes ${edad} años.`);
}

saludar("Ana", 30); // Output: Hola Ana, tienes 30 años.
```

nombre y **edad** son parámetros obligatorios.

La función **saludar** requiere que ambos parámetros se proporcionen al momento de la llamada.

2.- PARÁMETROS OPCIONALES

PARÁMETROS OPCIONALES – aquellos que no es necesario proporcionar un valor para ellos al llamar a la función.

Para hacer un parámetro opcional, simplemente agregamos un signo de interrogación (**?**) después del nombre del parámetro.

```
function saludar(nombre: string, edad?: number): void {
    if (edad) {
        console.log(`Hola ${nombre}, tienes ${edad} años.`);
    } else {
        console.log(`Hola ${nombre}.`);
    }
}
```

La función **saludar** recibe un parámetro obligatorio **nombre** de tipo string y un parámetro opcional **edad** de tipo number.

Si se proporciona un valor para **edad**, se mostrará un mensaje personalizado que incluye la edad.

Si no se proporciona un valor para **edad**, se mostrará un mensaje genérico.

3.- PARÁMETROS POR DEFECTO

PARÁMETROS POR DEFECTO – aquellos que, si no se proporciona un valor para un parámetro, se utilizará el valor por defecto especificado.

Para asignar un valor por defecto a un parámetro, utilizamos el operador de asignación (=) después del tipo de dato.

```
function saludar(nombre: string, edad: number = 18): void {
    console.log(`Hola ${nombre}, tienes ${edad} años.`);
}
```

El parámetro **edad** tiene un valor por defecto de 18. Si no se proporciona un valor para edad al llamar a la función, se utilizará el valor por defecto.

4.- PARÁMETROS REST

PARÁMETROS REST– aquellos que, permiten a una función aceptar un número variable de argumentos. Se definen utilizando los tres puntos (**...**) antes del nombre del parámetro y se recogen en un array.

```
function sumar(...numeros: number[]): number {
    return numeros.reduce((acc, curr) => acc + curr, 0);
}

console.log(sumar(1, 2, 3, 4)); // Output: 10
console.log(sumar(5, 10)); // Output: 15
```

numeros es un parámetro rest que permite a la función **sumar,** aceptar cualquier cantidad de argumentos numéricos y devolver su suma.

5.- PARÁMETROS CON TIPOS LITERALES

PARÁMETROS CON TIPOS LITERALES – aquellos que, sólo aceptan valores específicos usando tipos literales. Esto es útil cuando deseas restringir los valores permitidos para un parámetro.

```
function configurarModo(modo: "light" | "dark"): void {
    console.log(`Modo seleccionado: ${modo}`);
}

configurarModo("light"); // Output: Modo seleccionado: light
configurarModo("dark"); // Output: Modo seleccionado: dark
```

El parámetro **modo** sólo puede aceptar los valores "light" o "dark", lo que asegura que sólo se puedan usar estos dos valores específicos.

6.- PARÁMETROS DE FUNCIÓN COMO TIPO

PARÁMETROS DE FUNCIÓN COMO TIPO – TypeScript permite usar tipos de función como parámetros, lo que permite pasar funciones como argumentos a otras funciones.

```
function ejecutarOperacion(a: number, b: number, operacion: (x: number, y: number) => number): number {
    return operacion(a, b);
}

const suma = (x: number, y: number): number => x + y;
const resta = (x: number, y: number): number => x - y;

console.log(ejecutarOperacion(5, 3, suma)); // Output: 8
console.log(ejecutarOperacion(5, 3, resta)); // Output: 2
```

ejecutarOperacion acepta dos números y una función **operacion** que define cómo se deben combinar esos números.

14. Funciones y métodos

RETORNO – valor que devuelve una función después de ejecutar su bloque de código.

En TypeScript, se puede especificar el tipo de retorno de una función para asegurarse de que la función devuelve el tipo de dato esperado. Esto ayuda a prevenir errores y mejora la legibilidad del código.

```
function sumar(a: number, b: number): number {
    return a + b;
}
```

number es el tipo de retorno de la función sumar.

TypeScript permite especificar distintos tipos de retorno como: primitivos, complejos, sin retorno u opcionales.

TIPOS DE RETORNO PRIMITIVOS

TypeScript permite especificar tipos de retorno primitivos (como number, string, boolean, etc).

```
function obtenerNombre(): string {
    return "Luis";
}

function esMayorDeEdad(edad: number): boolean {
    return edad >= 18;
}
```

TIPOS DE RETORNO COMPLEJOS

Las funciones en TypeScript también pueden devolver tipos complejos (como objetos, arrays, o tuplas).

Esto es muy útil como alternativa para devolver múltiples valores de retorno. Dado que únicamente podemos devolver un único valor, pero podemos devolver una agrupación cualquier de valores.

```
function crearPersona(nombre: string, edad: number): { nombre: string; edad: number } {
    return { nombre, edad };
}

const persona = crearPersona("Ana", 25);
console.log(persona); // { nombre: 'Ana', edad: 25 }
```

OBJETOS

```
function obtenerNumeros(): number[] {
    return [1, 2, 3, 4, 5];
}

const numeros = obtenerNumeros();
console.log(numeros); // [1, 2, 3, 4, 5]
```

ARRAYS

```
type Persona = {
    nombre: string;
    edad: number;
};

function crearPersona(nombre: string, edad: number): Persona {
    return { nombre, edad };
}

const persona = crearPersona("Carlos", 30);
console.log(persona); // { nombre: 'Carlos', edad: 30 }
```

TIPOS PERSONALIZADOS

FUNCIONES QUE NO RETORNAN VALORES

En TypeScript, si una función no devuelve ningún valor, se utiliza el tipo **void** como tipo de retorno.

Esto es útil para funciones que solo realizan acciones y no necesitan devolver un valor.

```
function saludar(nombre: string): void {
    console.log(`Hola, ${nombre}`);
}

saludar("María"); // Hola, María
```

RETORNOS OPCIONALES

A veces, una función puede devolver un valor opcionalmente. Es decir, puede devolver un valor o undefined. En estos casos, se puede usar una unión de tipos (type union).

```
function encontrarElemento(arr: number[], valor: number): number | undefined {
    return arr.find((el) => el === valor);
}

const resultado = encontrarElemento([1, 2, 3], 2);
console.log(resultado); // 2

const resultadoInexistente = encontrarElemento([1, 2, 3], 4);
console.log(resultadoInexistente); // undefined
```

14. Funciones y métodos

SOBRECARGA DE FUNCIONES – permite declarar varias versiones de una función con diferentes parámetros y tipos de retorno. Cada versión de la función (o firma) se llama **sobrecarga**.

La implementación real de la función debe manejar todas las combinaciones de parámetros definidos en las sobrecargas.

Para crear sobrecargas de funciones en TypeScript debemos hacer lo siguiente

1) Definir las firmas de las funciones sobrecargadas.
2) Proveer una única implementación de la función que maneje todas las combinaciones de parámetros.

```
// Definición de sobrecargas
function miFuncion(param: string): string;
function miFuncion(param: number): number;

// Implementación de la función
function miFuncion(param: string | number): string | number {
    // hacer cosas

    return "";
}
```

miFuncion tiene dos firmas: una que acepta un string y otra que acepta un number. La implementación de la función maneja ambas firmas.

14. Funciones y métodos

Si intentamos llamar a la función con un tipo, o una cantidad de parámetros que no están admitidas por las firmas, recibiremos un error del compilador o del IDE.

```
miFuncion(12);   // esto no da error
miFuncion("12")  // esto no da error

let miObjeto = {};
miFuncion(miObjeto);  // esto da un error, no admite un objeto

miFuncion(12, 12);    // esto da un error, no admite dos int
```

Cuando existen firmas sobrecargadas de una función, TypeScript convierte la función que define el objeto en privada. Por tanto, no podemos usarla directamente, y debemos usar obligatoriamente una de sus firmas definidas.

SOBRECARGA CON DIFERENTES TIPOS DE PARÁMETROS

```
// Definición de sobrecargas
function combinar(a: string, b: string): string;
function combinar(a: number, b: number): number;
// Implementación de la función
function combinar(a: string | number, b: string | number): string | number {
    if (typeof a === "string" && typeof b === "string") {
        return a + b;
    } else if (typeof a === "number" && typeof b === "number") {
        return a + b;
    }
    throw new Error("Tipos de parámetros no coinciden");
}
console.log(combinar("Hola, ", "mundo")); // "Hola, mundo"
console.log(combinar(5, 10)); // 15
```

La implementación de **combinar** admite tanto string como number como parámetros, para cumplir con ambas definiciones.

Alternativamente, podríamos haber usado **any**. Aunque en la medida de lo posible, por limpieza, es mejor que especifiquemos los tipos.

SOBRECARGA CON DIFERENTES CANTIDADES DE PARÁMETROS

También es posible sobrecargar funciones con diferentes cantidades de parámetros.

```
// Definición de sobrecargas
function mostrarMensaje(mensaje: string): void;
function mostrarMensaje(mensaje: string, veces: number): void;

// Implementación de la función
function mostrarMensaje(mensaje: string, veces?: number): void {
    if (veces === undefined) {
        console.log(mensaje);
    } else {
        for (let i = 0; i < veces; i++) {
            console.log(mensaje);
        }
    }
}

mostrarMensaje("Hola"); // "Hola"
mostrarMensaje("Hola", 3); // "Hola" "Hola" "Hola"
```

veces es un parámetro opcional que permite a la función **mostrarMensaje** manejar uno o dos argumentos.

14. Funciones y métodos

TypeScript admite **funciones anónimas** y **funciones de flecha**, que son formas más concisas de escribir funciones.

FUNCIONES ANÓNIMAS

FUNCIONES ANÓNIMAS - son funciones sin nombre, comúnmente utilizadas como argumentos de otras funciones.

```
const saludar = function(nombre: string): void {
    console.log(`Hola ${nombre}.`);
};
```

Estas funciones (como su nombre indica) no tienen un nombre asignado y se utilizan en una variedad de contextos. Especialmente cuando se necesita una función temporal.

FUNCIONES FLECHA

FUNCIONES FLECHA - sintaxis compacta de definir funciones en JavaScript y TypeScript. Se caracterizan por usar la sintaxis **=>** y tienen varias ventajas sobre las funciones tradicionales, ya que permiten escribir funciones de manera más concisa y legible, especialmente cuando se trata de funciones cortas o funciones de una sola línea.

```
const saludar = (nombre: string): void => {
    console.log(`Hola ${nombre}.`);
};
```

Las funciones de flecha son especialmente útiles en contextos donde se necesita preservar el contexto **this** de la función envolvente, ya que utilizan el contexto **this** del entorno en el que fueron creadas.

14. Funciones y métodos

DECORADORES - son una sintaxis para funciones especiales que se aplican a una clase, método, propiedad o parámetro para modificar o añadir comportamientos adicionales.

Estas funciones se ejecutan en tiempo de compilación y pueden realizar tareas como la modificación de la funcionalidad de un método, el seguimiento de las llamadas a un método, la validación de parámetros de entrada, etc.

Para crear un **decorador** en Typescript, simplemente necesitamos definir una función con el nombre del decorador y aplicarla a la clase, método, propiedad o parámetro que queramos decorar.

```
function logMethod(target: any, key: string, descriptor: PropertyDescriptor) {
  const originalMethod = descriptor.value;

  descriptor.value = function (...args: any[]) {
    console.log(`Método ${key} llamado con argumentos ${args}`);
    return originalMethod.apply(this, args);
  };

  return descriptor;
}
```

Definimos una función llamada **logMethod** que toma tres parámetros: **target** (clase a la que se aplica el decorador), **key** (nombre del método decorado) y **descriptor** (objeto que contiene información sobre el método decorado).

En el cuerpo de esta función: guardamos una referencia al método original, redefinimos el método para que imprima un mensaje de registro antes de llamar al método original y finalmente devolvemos el descriptor modificado.

14. Funciones y métodos

Una vez que hemos creado nuestro **decorador**, podemos aplicarlo a la clase, método, propiedad o parámetro que deseemos decorar utilizando el símbolo **@** seguido del nombre del decorador.

```
class MyClass {
  @logMethod
  myMethod(arg1: string, arg2: number) {
    console.log(`Método myMethod llamado con argumentos ${arg1} y ${arg2}`);
  }
}
```

Aplicamos el decorador **logMethod** al método **myMethod** de la clase MyClass

Cuando se llame a **myMethod**, el decorador imprimirá un mensaje de registro antes de ejecutar el código original del método.

15. Clases y objetos

Las **clases** en TypeScript proporcionan una manera estructurada para definir objetos con propiedades y métodos asociados.

Las **clases** son fundamentales en la programación orientada a objetos, ya que nos permiten definir objetos que encapsulan datos y comportamientos relacionados.

En TypeScript, puedes definir una clase utilizando la palabra clave **class**, seguida del nombre de la clase y un bloque de código que contiene sus miembros.

```
class Persona {
    nombre: string;
    edad: number;

    constructor(nombre: string, edad: number) {
        this.nombre = nombre;
        this.edad = edad;
    }

    saludar(): void {
        console.log(`Hola, mi nombre es ${this.nombre} y tengo ${this.edad} años.`);
    }
}
```

class Persona: Define una nueva clase llamada Persona.

nombre: string y **edad: number** - Son propiedades de la clase. Se definen con un tipo de dato que especifica qué tipo de valores pueden almacenar.

constructor(nombre: string, edad: number) - Es un método especial llamado constructor que se invoca al crear una nueva instancia de la clase. Inicializa las propiedades nombre y edad.

saludar(): void - Es un método que no devuelve ningún valor (void) y que imprime un saludo en la consola.

INSTANCIAR UNA CLASE

Una vez que tenemos una clase definida, podemos crear instancias de ella utilizando la palabra clave **new**.

Cada instancia será un objeto separado que tiene sus propias propiedades y métodos.

Por ejemplo, vamos a usar nuestra clase Persona para definir dos instancias

```
const persona1 = new Persona("Luis", 30);
persona1.saludar();  // Salida: Hola, mi nombre es Luis y tengo 30 años.

const persona2 = new Persona("Ana", 25);
persona2.saludar();  // Salida: Hola, mi nombre es Ana y tengo 25 años.
```

new Persona("Luis", 30) - Crea una nueva instancia de la clase Persona, llamando al constructor con los argumentos "Luis" y 30.

persona1.saludar() - Llama al método saludar del objeto persona1, que imprime un mensaje de saludo usando los valores de sus propiedades.

Y lo mismo para ocurriría para el caso de Ana.

PROPIEDADES Y MÉTODOS

Las propiedades y métodos de una clase se pueden definir en la clase misma, añadiéndolos en el cuerpo de la clase.

```
class Coche {
    marca: string;
    modelo: string;
    anio: number;

    constructor(marca: string, modelo: string, anio: number) {
        this.marca = marca;
        this.modelo = modelo;
        this.anio = anio;
    }

    describir(): void {
        console.log(`El coche es un ${this.marca} ${this.modelo} del año ${this.anio}.`);
    }
}

const miCoche = new Coche("Toyota", "Corolla", 2021);
miCoche.describir();  // Salida: El coche es un Toyota Corolla del año 2021.
```

CONSTRUCTOR

El constructor es un método especial que se invoca automáticamente al crear una nueva instancia de una clase.

Su propósito principal es inicializar las propiedades del objeto con los valores proporcionados durante la instanciación.

La **clase Libro** define un constructor que recibe dos parámetros: **titulo**, **autor**.

Estos parámetros se utilizan para inicializar las propiedades correspondientes de la clase.

```
class Libro {
    titulo: string;
    autor: string;

    constructor(titulo: string, autor: string) {
        this.titulo = titulo;
        this.autor = autor;
    }
}

const libro1 = new Libro("Cien años de soledad", "Gabriel García Márquez");
```

FORMA SIMPLIFICADA

TypeScript ofrece una forma más concisa de definir constructores utilizando modificadores de acceso directamente en los parámetros del constructor.

```
class Libro {
    constructor(public titulo: string, public autor: string) {
        }
}

const libro1 = new Libro("Cien años de soledad", "Gabriel García Márquez");
console.log(libro1);
```

Esta forma simplifica el código al combinar la declaración y la inicialización de las propiedades en una sola línea

MODIFICADORES DE ACCESO

MODIFICADORES DE ACCESO - son palabras clave que se utilizan para controlar la visibilidad y accesibilidad de las propiedades y métodos dentro de una clase.

Estos modificadores permiten definir qué partes de una clase pueden ser accedidas desde fuera de la clase y cuáles deben permanecer ocultas, facilitando la encapsulación y el diseño más seguro del código.

Los tres principales modificadores de acceso disponibles en TypeScript son:

Modificador	Descripción
public	Permite el acceso a propiedades y métodos desde cualquier parte del código. Es el modificador por defecto si no se especifica otro.
private	Restringe el acceso a propiedades y métodos a la propia clase, ocultándolos de clases derivadas y del resto del código.
protected	Permite el acceso a propiedades y métodos dentro de la propia clase y sus clases derivadas, pero no desde fuera de ellas.

MÉTODOS Y VARIABLES ESTÁTICOS

MÉTODOS Y VARIABLES ESTÁTICOS - son aquellos que pertenecen a la clase en sí misma en lugar de a una instancia específica de la clase.

Se utilizan para definir comportamientos que están relacionados con la clase en su conjunto, y no con instancias individuales.

MÉTODO ESTÁTICO – método que se define con la palabra clave **static**.

```
class Calculadora {
    static sumar(a: number, b: number): number {
        return a + b;
    }

    static restar(a: number, b: number): number {
        return a - b;
    }
}

// Uso de los métodos estáticos
console.log(Calculadora.sumar(5, 3)); // Salida: 8
console.log(Calculadora.restar(5, 3)); // Salida: 2
```

sumar y **restar** son métodos estáticos que realizan operaciones matemáticas. Se accede a ellos directamente desde la clase **Calculadora**, sin necesidad de crear una instancia.

Los métodos estáticos no tienen acceso a las propiedades de instancia (this), ya que están asociados a la clase en sí misma

15. Clases y objetos

VARIABLE ESTÁTICA – variable que pertenece a la clase en lugar de a instancias individuales.

Las variables estáticas se pueden usar para almacenar valores que son comunes para todas las instancias de la clase.

```
class Configuracion {
    static readonly API_URL: string = "https://api.ejemplo.com";
    static readonly MAX_INTENTOS: number = 5;
}

// Uso de las variables estáticas
console.log(Configuracion.API_URL);         // Salida: https://api.ejemplo.com
console.log(Configuracion.MAX_INTENTOS);    // Salida: 5
```

PI_URL y **MAX_INTENTOS** son variables estáticas que se definen dentro de la clase **Configuracion**.

Estas variables se pueden acceder directamente a través de la clase, sin necesidad de instanciarla.

16. Interfaces

INTERFACE – herramienta que permite definir la estructura y el comportamiento de un objeto especificando los nombres, los tipos de sus propiedades y métodos.

Posteriormente, las clases pueden implementar este interface (en este caso, deben definir todas las variables y métodos que incluya el interfaz).

Para declarar una interfaz en Typescript, utilizamos la palabra clave **interface** seguida del nombre de la interfaz y las propiedades y métodos que queremos definir.

```
interface Persona {
  nombre: string;
  edad: number;
  saludar(): void;
}
```

Hemos declarado una interfaz llamada **Persona** que define tres propiedades:

- **nombre** de tipo string
- **edad** de tipo number
- **saludar** que es un método sin retorno (void).

16. Interfaces

Una vez que hemos declarado una interfaz, podemos implementarla en una clase utilizando la palabra clave **implements**.

Esto establece que la clase debe implementar las variables y métodos declarados por la interfaz.

```
class Estudiante implements Persona {
  nombre: string;
  edad: number;

  constructor(nombre: string, edad: number) {
    this.nombre = nombre;
    this.edad = edad;
  }

  saludar() {
    console.log(`Hola, mi nombre es ${this.nombre} y tengo ${this.edad} años.`);
  }
}
```

Hemos declarado una clase llamada **Estudiante** que implementa la interfaz **Persona**.

La clase **Estudiante** debe definir las propiedades **nombre** y **edad**, así como el método **saludar**, tal como se define en la interfaz Persona.

EXTENSIÓN DE INTERFACES

TypeScript permite la extensión de interfaces, permitiendo crear interfaces más específicos basados en otros interfaces.

```
interface Animal {
    nombre: string;
}

interface Mascota extends Animal {
    vacunas: boolean;
}

const mascota: Mascota = { nombre: "Firulais", vacunas: true };
console.log(mascota); // { nombre: "Firulais", vacunas: true }
```

Mascota extiende **Animal**, agregando la propiedad **vacunas**.

Cualquier objeto que implemente **Mascota** también debe cumplir con las propiedades **de Animal**.

PROPIEDADES OPCIONALES

Se definen añadiendo un signo de interrogación (**?**) después del nombre de la propiedad. Esto indica que la propiedad puede no estar presente en el objeto.

```
interface Libro {
    titulo: string;
    autor?: string;
}

const libro1: Libro = { titulo: "TypeScript Básico" };
const libro2: Libro = { titulo: "TypeScript Avanzado", autor: "Luis Pérez" };
```

PROPIEDADES DE SÓLO LECTURA

Se definen usando la palabra clave **readonly**. Esto asegura que la propiedad no puede ser modificada después de su inicialización.

```
interface Punto {
    readonly x: number;
    readonly y: number;
}

const punto: Punto = { x: 10, y: 20 };
// punto.x = 5; // Error: no se puede asignar a 'x' porque es una propiedad de solo lectura.
```

17. Herencia y polimorfismo

HERENCIA - es un mecanismo que nos permite definir una nueva clase basada en una clase existente. La clase existente se conoce como la **clase base** o **superclase**, y la nueva clase se conoce como la **clase derivada** o **subclase**.

La subclase hereda todas las propiedades y métodos de la superclase y puede agregar nuevos atributos y comportamientos según sea necesario.

En TypeScript, podemos lograr la herencia utilizando la palabra clave **extends**. La clase base define las propiedades y métodos comunes, mientras que la clase derivada extiende y especializa estos miembros.

```
// Clase base
class Animal {
    constructor(public nombre: string) {}

    hacerSonido(): void {
        console.log(`${this.nombre} hace un sonido.`);
    }
}

// Clase derivada
class Perro extends Animal {
    ladrar(): void {
        console.log(`${this.nombre} está ladrando.`);
    }
}

const miPerro = new Perro("MiPerro");
miPerro.hacerSonido(); // Salida: MiPerro hace un sonido.
miPerro.ladrar();      // Salida: MiPerro está ladrando.
```

Perro hereda la propiedad **nombre** y el método **hacerSonido** de la clase **Animal**

Añade su propio método ladrar

SOBRESCRITURA DE MÉTODOS

Una clase derivada puede sobrescribir métodos de la clase base utilizando la palabra clave **override**.

```
class Animal {
    constructor(public nombre: string) {}

    hacerSonido(): void {
        console.log(`${this.nombre} hace un sonido.`);
    }
}

class Gato extends Animal {
    override hacerSonido(): void {
        console.log(`${this.nombre} maúlla.`);
    }
}

const miGato = new Gato("Misi");
miGato.hacerSonido(); // Salida: Misi maúlla.
```

El método **hacerSonido** en **Gato** sobrescribe la implementación del método **hacerSonido** en **Animal**.

17. Herencia y polimorfismo

POLIMORFISMO - permite tratar objetos de diferentes clases de manera uniforme, siempre y cuando compartan una interfaz o superclase común.

En Typescript, podemos lograr el polimorfismo utilizando interfaces.

```
interface Animal {
  hacerSonido(): void;
}
class Dog implements Animal {
  hacerSonido() {
    console.log("El perro ladra");
  }
}
class Cat implements Animal {
  hacerSonido() {
    console.log("El gato maulla");
  }
}
function makeAnimalSound(animal: Animal) {
  animal.hacerSonido();
}
const myDog = new Dog();
const myCat = new Cat();
makeAnimalSound(myDog); // Output: "El perro ladra"
makeAnimalSound(myCat); // Output: "El gato maulla"
```

Creamos una interfaz **Animal** con un método **hacerSonido**.

Luego, creamos las clases **Dog** y **Cat** que implementan la interfaz **Animal**.

Finalmente, creamos una función **makeAnimalSound** que acepta un parámetro de tipo **Animal** y llama al método **hacerSonido** del objeto pasado.

EJERCICIO 6

Crea interface Persona (nombre) y la interface Empleado que extiende de Persona (salario).

SOLUCIÓN:

```
1  interface Persona {
2    nombre: string;
3  }
4
5  interface Empleado extends Persona {
6    salario: number;
7  }
8
9  const e1: Empleado = { nombre: "Ana", salario: 1500 };
10 console.log(e1);
```

EJERCICIO 7

Crea un enum Rol con "Admin", "Editor", "Visitante", y una función que de permisos según el rol.

De tal forma que los permisos son:

a) Admin – leer, escribir y borrar

b) Editor – leer y escribir

c) Visitante - leer

SOLUCIÓN:

```
1  enum Rol {
2    Admin = "Admin",
3    Editor = "Editor",
4    Visitante = "Visitante",
5  }
6
7  function permisos(rol: Rol): string[] {
8    switch (rol) {
9      case Rol.Admin:
10       return ["leer", "escribir", "borrar"];
11     case Rol.Editor:
12       return ["leer", "escribir"];
13     default:
14       return ["leer"];
15   }
16 }
17
18 console.log(permisos(Rol.Editor));
```

EJERCICIO 8

Crea en TypeScript una función asíncrona que devuelva un número después de 1 segundo.

SOLUCIÓN:

```
function esperar(): Promise<number> {
  return new Promise(resolve => {
    setTimeout(() => resolve(42), 1000);
  });
}

async function demo() {
  const valor = await esperar();
  console.log(valor);
}

demo();
```

EJERCICIO 9

Crea una función imprimirFecha que acepte Date | string y devuelva siempre una fecha formateada. .

SOLUCIÓN:

```
1  function imprimirFecha(valor: Date | string): string {
2    if (valor instanceof Date) {
3      return valor.toISOString();
4    }
5    return new Date(valor).toISOString();
6  }
7
8  console.log(imprimirFecha("2020-01-01"));
9  console.log(imprimirFecha(new Date()));
```

EJERCICIO 10

Crea una función asíncrona que devuelva un objeto { ok: boolean; tiempo: number } después de 500ms.

SOLUCIÓN:

```typescript
1  async function medirTiempo(): Promise<{ ok: boolean; tiempo: number }> {
2    return new Promise(resolve =>
3      setTimeout(() => resolve({ ok: true, tiempo: 500 }), 500)
4    );
5  }
6
7  medirTiempo().then(console.log);
```

EJERCICIO 11

a) Crea una entidad Libro con propiedades: titulo, autor y paginas

b) Añade a Libro un método describir() que devuelva texto formateado.

c) Haz que Libro lance un error si paginas <= 0.

SOLUCIÓN:

a)

```typescript
class Libro {
  constructor(
    public titulo: string,
    public autor: string,
    public paginas: number
  ) {}
}

const l1 = new Libro("1984", "George Orwell", 328);
console.log(l1);
```

b)

```typescript
class Libro {
  constructor(
    public titulo: string,
    public autor: string,
    public paginas: number
  ) {}

  describir(): string {
    return `${this.titulo} – ${this.autor} (${this.paginas} páginas)`;
  }
}

const libro = new Libro("El Hobbit", "Tolkien", 310);
console.log(libro.describir());
```

c)

```typescript
class Libro {
  constructor(
    public titulo: string,
    public autor: string,
    public paginas: number
  ) {
    if (paginas <= 0) {
      throw new Error("El número de páginas debe ser mayor que 0");
    }
  }
}

const l = new Libro("Clean Code", "Robert C. Martin", 450);
console.log(l);
```

EJERCICIO 12

Crea una entidad Cliente bancario que incluya: id, nombre, dni, email y teléfono (opcional).

Añade una función para darlo de alta.

NOTA: utilizar para id crypto.randomUUID() (generador de números aleatorios criptográficamente seguro, devuelve un UUID v4 que tiene el formato: xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx)

```
{id: '0a9e65a6-2b6e-4ce2-a43c-1e306bc802a9', nombre: 'Ana López', dni: '12345678A',
email: 'ana@example.com', telefono: undefined} i
  dni: "12345678A"
  email: "ana@example.com"
  id: "0a9e65a6-2b6e-4ce2-a43c-1e306bc802a9"
  nombre: "Ana López"
  telefono: undefined
```

EJERCICIO 12 – SOLUCIÓN:

```typescript
// Entidad (interface) Cliente
interface Cliente {
  id: string;          // p.ej. UUID
  nombre: string;
  dni: string;
  email: string;
  telefono?: string;   // opcional
}

// Función que crea un cliente a partir de datos
function crearCliente(
  nombre: string,
  dni: string,
  email: string,
  telefono?: string
): Cliente {
  const id = crypto.randomUUID();
  return {
    id,
    nombre,
    dni,
    email,
    telefono,
  };
}

// Ejemplo de uso
const c1 = crearCliente("Ana López", "12345678A", "ana@example.com");
console.log(c1);
```

EJERCICIO 13

Utilizando el cliente del ejercicio anterior.

Crea una entidad llamada CuentaBancaria con: iban (sólo lectura), saldo, idCliente (sólo lectura, se extrae del ejercicio anterior) y Estado de la cuenta.

Estado de la cuenta puede ser: activada, bloqueada o cerrada.

Implementa métodos para ingresar y retirar saldo de la cuenta.

```
Ej02.ts:29
▾ {id: 'ca8f2d9d-146b-410d-b868-bff1ea68afdc', nombre: 'Ana López', dni: '12345678A', email:
  'ana@example.com', telefono: undefined} i
    dni: "12345678A"
    email: "ana@example.com"
    id: "ca8f2d9d-146b-410d-b868-bff1ea68afdc"
    nombre: "Ana López"
    telefono: undefined
  ▸ [[Prototype]]: Object
Ej02.ts:77
▾ CuentaBancaria {iban: 'ES76 1234 5678 0000 0001', saldo: 1000, idCliente: 'ca8f2d9d-146b-41
  0d-b868-bff1ea68afdc', estado: 'ACTIVA'} i
    estado: "ACTIVA"
    iban: "ES76 1234 5678 0000 0001"
    idCliente: "ca8f2d9d-146b-410d-b868-bff1ea68afdc"
    saldo: 1300
  ▸ [[Prototype]]: Object
Saldo Inicial:  1000                                   Ej02.ts:78
Saldo +500€:  1500                                     Ej02.ts:80
Saldo -200€:  1300                                     Ej02.ts:82
Saldo final:  1300                                     Ej02.ts:83
```

18. Ejercicios TypeScript

EJERCICIO 13 – SOLUCIÓN:

```typescript
// Tipo para el estado posible de una cuenta
type EstadoCuenta = "ACTIVA" | "BLOQUEADA" | "CERRADA";
// Entidad CuentaBancaria
class CuentaBancaria {
  public readonly iban: string;
  public saldo: number;
  public readonly idCliente: string;
  public estado: EstadoCuenta;
  constructor(iban: string, idCliente: string, saldoInicial: number = 0) {
    this.iban = iban;
    this.idCliente = idCliente;
    this.saldo = saldoInicial;
    this.estado = "ACTIVA";
  }
  ingresar(cantidad: number): void {
    if (this.estado !== "ACTIVA") {
      throw new Error("La cuenta no está activa.");
    }
    if (cantidad <= 0) {
      throw new Error("La cantidad a ingresar debe ser positiva.");
    }
    this.saldo += cantidad;
  }
  retirar(cantidad: number): void {
    if (this.estado !== "ACTIVA") {
      throw new Error("La cuenta no está activa.");
    }
    if (cantidad <= 0) {
      throw new Error("La cantidad a retirar debe ser positiva.");
    }
    if (cantidad > this.saldo) {
      throw new Error("Saldo insuficiente.");
    }
    this.saldo -= cantidad;
  }
  bloquear(): void {
    this.estado = "BLOQUEADA";
  }
  cerrar(): void {
    if (this.saldo !== 0) {
      throw new Error("No se puede cerrar una cuenta con saldo distinto de 0.");
    }
    this.estado = "CERRADA";
  }
}
// Ejemplo de uso
const cuentaAna = new CuentaBancaria("ES76 1234 5678 0000 0001", c1.id, 1000);
console.log(cuentaAna);
console.log("Saldo Inicial: ", cuentaAna.saldo);
cuentaAna.ingresar(500);
console.log("Saldo +500€: ", cuentaAna.saldo);
cuentaAna.retirar(200);
console.log("Saldo -200€: ", cuentaAna.saldo);
console.log("Saldo final: ", cuentaAna.saldo);
```

```
Ej02.ts:29
{id: 'ca8f2d9d-146b-410d-b868-bff1ea68afdc', nombre: 'Ana López', dni: '12345678A', email: 'ana@example.com', telefono: undefined}
  dni: "12345678A"
  email: "ana@example.com"
  id: "ca8f2d9d-146b-410d-b868-bff1ea68afdc"
  nombre: "Ana López"
  telefono: undefined
  [[Prototype]]: Object
Ej02.ts:77
CuentaBancaria {iban: 'ES76 1234 5678 0000 0001', saldo: 1000, idCliente: 'ca8f2d9d-146b-410d-b868-bff1ea68afdc', estado: 'ACTIVA'}
  estado: "ACTIVA"
  iban: "ES76 1234 5678 0000 0001"
  idCliente: "ca8f2d9d-146b-410d-b868-bff1ea68afdc"
  saldo: 1300
  [[Prototype]]: Object
Saldo Inicial:  1000                Ej02.ts:78
Saldo +500€:  1500                  Ej02.ts:80
Saldo -200€:  1300                  Ej02.ts:82
Saldo final:  1300                  Ej02.ts:83
```

EJERCICIO 14

Utilizando el cliente y la cuenta bancaria de los ejercicios anteriores.

Crea una entidad llamada Movimiento con: id, fecha, tipo de movimiento, concepto, importe, saldoPosterior.

El tipo movimiento puede ser: ingreso, retirada, transferencia_entrante, transferencia_saliente.

Crear una clase CuentaConMovimiento que extiende de CuentaBancaria, debe permitir: registrar un movimiento (ingresar o retirar)

```
▼ (2) [{…}, {…}] i
  ▼ 0:
      concepto: "Ingreso en efectivo"
    ▶ fecha: Thu Feb 19 2026 21:41:00 GMT+0100 (hora estándar de Europa central) {}
      id: "ce49d53d-cb1e-43a5-82cf-7c721fcfb0af"
      importe: 1000
      saldoPosterior: 1000
      tipo: "INGRESO"
    ▶ [[Prototype]]: Object
  ▼ 1:
      concepto: "Retirada en efectivo"
    ▶ fecha: Thu Feb 19 2026 21:41:00 GMT+0100 (hora estándar de Europa central) {}
      id: "b055e864-0bd1-43dc-84fa-df29b44a3763"
      importe: -250
      saldoPosterior: 750
      tipo: "RETIRADA"
    ▶ [[Prototype]]: Object
    length: 2
  ▶ [[Prototype]]: Array(0)
```

EJERCICIO 14 – SOLUCIÓN:

```typescript
type TipoMovimiento = "INGRESO" | "RETIRADA" | "TRANSFERENCIA_ENTRANTE" | "TRANSFERENCIA_SALIENTE";
interface Movimiento {
  id: string;
  fecha: Date;
  tipo: TipoMovimiento;
  concepto: string;
  importe: number;
  saldoPosterior: number;
}
// Extensión simple de CuentaBancaria para guardar movimientos
class CuentaConMovimientos extends CuentaBancaria {
  private movimientos: Movimiento[] = [];
  private registrarMovimiento(
    tipo: TipoMovimiento,
    concepto: string,
    importe: number
  ): void {
    const movimiento: Movimiento = {
      id: crypto.randomUUID(),
      fecha: new Date(),
      tipo,
      concepto,
      importe,
      saldoPosterior: this.saldo,
    };
    this.movimientos.push(movimiento);
  }
  override ingresar(cantidad: number): void {
    super.ingresar(cantidad);
    this.registrarMovimiento("INGRESO", "Ingreso en efectivo", cantidad);
  }
  override retirar(cantidad: number): void {
    super.retirar(cantidad);
    this.registrarMovimiento("RETIRADA", "Retirada en efectivo", -cantidad);
  }
  obtenerMovimientos(): Movimiento[] {
    return [...this.movimientos]; // copia defensiva
  }
}
// Ejemplo
const cuentaMov = new CuentaConMovimientos("ES76 1234 5678 0000 0002", c1.id, 0);
cuentaMov.ingresar(1000);
cuentaMov.retirar(250);
console.log(cuentaMov.obtenerMovimientos());
```

EJERCICIO 15

Utilizando las interfaces de los ejercicios anteriores.

Crea un servicio que gestiona transferencias entre dos cuentas.

Implementa la función transferir con: origen, destino, cantidad, concepto y el resultado de la transferencia.

El resultado de transferencia es un interface con: ok y mensaje.

```
Cuenta origen                                                                    Ej04.ts:167
 ▾ CuentaConMovimientos {iban: 'ES76 0000 0000 0000 0003', saldo: 300, idCliente: 'clienteA',
   estado: 'ACTIVA', movimientos: Array(2)} ℹ
     estado: "ACTIVA"
     iban: "ES76 0000 0000 0000 0003"
     idCliente: "clienteA"
   ▸ movimientos: (2) [{…}, {…}]
     saldo: 300
   ▸ [[Prototype]]: CuentaBancaria
Cuenta destino
 ▾ CuentaConMovimientos {iban: 'ES76 0000 0000 0000 0004', saldo: 300, idCliente: 'clienteB',
   estado: 'ACTIVA', movimientos: Array(2)} ℹ
     estado: "ACTIVA"
     iban: "ES76 0000 0000 0000 0004"
     idCliente: "clienteB"
   ▸ movimientos: (2) [{…}, {…}]
     saldo: 300
   ▸ [[Prototype]]: CuentaBancaria
Cantidad:  200 Concepto: Pago alquiler
 ▾ {ok: true, mensaje: 'Transferencia realizada correctamente'} ℹ                Ej04.ts:168
     mensaje: "Transferencia realizada correctamente"
     ok: true
   ▸ [[Prototype]]: Object
Saldo de la cuenta A: 300                                                        Ej04.ts:169
Saldo B: 300                                                                     Ej04.ts:170
```

EJERCICIO 15 – SOLUCIÓN:

```
128  interface ResultadoTransferencia {
129    ok: boolean;
130    mensaje: string;
131  }
132  function transferir(
133    origen: CuentaConMovimientos,
134    destino: CuentaConMovimientos,
135    cantidad: number,
136    concepto: string = "Transferencia"
137  ): ResultadoTransferencia {
138    if (origen === destino) {
139      return { ok: false, mensaje: "La cuenta origen y destino no pueden ser la misma." };
140    }
141    if (cantidad <= 0) {
142      return { ok: false, mensaje: "La cantidad debe ser positiva." };
143    }
144    try {
145      origen.retirar(cantidad);
146      // Registramos como transferencia saliente en origen
147      (origen as any).registrarMovimiento?.(
148        "TRANSFERENCIA_SALIENTE",
149        concepto,
150        -cantidad
151      );
152      destino.ingresar(cantidad);
153      // Registramos como transferencia entrante en destino
154      (destino as any).registrarMovimiento?.(
155        "TRANSFERENCIA_ENTRANTE",
156        concepto,
157        cantidad
158      );
159      return { ok: true, mensaje: "Transferencia realizada correctamente" };
160    } catch (error: any) {
161      return { ok: false, mensaje: error.message ?? "Error al realizar la transferencia." };
162    }
163  }
164  const cuentaA = new CuentaConMovimientos("ES76 0000 0000 0000 0003", "clienteA", 500);
165  const cuentaB = new CuentaConMovimientos("ES76 0000 0000 0000 0004", "clienteB", 100);
166  const resultado = transferir(cuentaA, cuentaB, 200, "Pago alquiler");
167  console.log("Cuenta origen", cuentaA, "Cuenta destino", cuentaB, "Cantidad: ", 200, "Concepto: Pago alquiler");
168  console.log(resultado);
169  console.log("Saldo de la cuenta A:", cuentaA.saldo);
170  console.log("Saldo B:", cuentaB.saldo);
```

Gracias